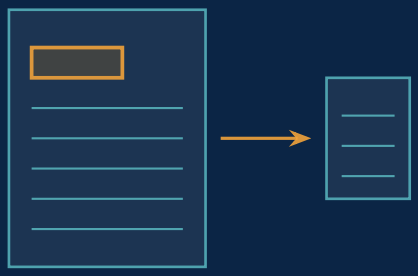
ICT UNIVERSITY · FACULTY OF ICT · DEPARTMENT OF COMPUTER SCIENCE



DocuMind AI

Intelligent Document Processing and Automated ERP Ingestion System

Phase 2 – System Design and Architecture

ARCHITECTURE · DATABASE · WORKFLOW · API · INTERFACE · SECURITY

Student	Tuheu Tchoubi Pempeme Moussa Fahdil
Programme	B.Sc. Computer Science, final-year project
Supervisor	[Supervisor name – to be completed]
Institution	The ICT University, Cameroon
Version	1.1 – October 2026

Document Control

Item	Description
Document	DocuMind AI – Phase 2: System Design and Architecture.
Version	1.1 (October 2026). Version 1.1 adds Chapter 9 (tender module), five tables and ten API operations.
Deliverable	Design specification covering user stories US-06 to US-11: system architecture, database design, document-processing workflow, REST API, user interface, security and ERP integration.
Companion files	`schema.sql` (MySQL schema), `openapi.yaml` (API contract) and `gen_tables.py` (generates the tables of this document from those two files).
Builds on	*Phase 1 Requirements Specification* (requirements FR-01 to FR-22, NFR-01 to NFR-10, business rules BR-1 to BR-6).
Status	**Draft for supervisor review.** Decisions that remain open are listed in Section 8.2 and Chapter 9.
Honesty note	This is a design. The two health endpoints are the only API code that exists. Section 1.4 states exactly what was verified and what was not.

Contents

Document Control i

1 Overview of Phase 2 1

1.1 Goal . . . 1
1.2 User stories covered . . . 1
1.3 Main design decisions at a glance . . . 1
1.4 What was checked . . . 2

2 System Architecture 3

2.1 Scope and context . . . 3
2.2 Layered architecture . . . 4
2.2.1 Technology decisions (acceptance criteria of US-06) . . . 4
2.3 Module rules . . . 5
2.3.1 Planned backend layout . . . 5
2.4 OCR pipeline and ERP integration in one view . . . 6
2.5 Deployment views . . . 6
2.6 Architecture decision records . . . 7

3 Database Design 8

3.1 Principles . . . 8
3.2 Entity-relationship diagram . . . 8
3.3 Purpose of each table . . . 9
3.4 Column dictionary . . . 9
3.5 Keys, constraints and indexes . . . 13
3.6 Audit design . . . 14
3.7 Status and history . . . 14
3.8 Verification performed . . . 14
3.9 From design to migrations . . . 15
3.10 Retention and privacy . . . 15

4 Document-Processing Workflow 16

4.1 Lifecycle . . . 16
4.2 Stage by stage . . . 17
4.3 File validation at upload . . . 18
4.4 Validation rules . . . 18
4.5 Confidence and review routing . . . 19

4.6 Failure and retry . . . 19
4.7 Upload sequence . . . 20

5 REST API Design **21**
5.1 Conventions . . . 21
5.2 Authentication and authorisation . . . 21
5.3 Endpoint catalogue . . . 22
5.4 Examples . . . 23
5.4.1 Sign in . . . 23
5.4.2 Upload . . . 23
5.4.3 Extraction result . . . 23
5.4.4 Correction . . . 24
5.5 Error model . . . 24
5.6 Idempotency and concurrency . . . 25
5.7 How the contract will be used . . . 25

6 User Interface Design **26**
6.1 Users and tasks . . . 26
6.2 Screen map . . . 26
6.3 Dashboard . . . 27
6.4 Upload page . . . 27
6.5 Extraction results and review . . . 28
6.6 Other screens . . . 28
6.7 Loading, success, empty and error states . . . 28
6.8 Frontend structure and usability . . . 29

7 Security and ERP Integration **30**
7.1 Threats and controls . . . 30
7.2 Credentials and configuration . . . 30
7.3 Data protection . . . 31
7.4 ERP integration . . . 31
7.4.1 Target and options . . . 31
7.4.2 Adapter interface . . . 31
7.4.3 Field mapping . . . 32
7.4.4 Submission flow and idempotency . . . 33
7.4.5 Errors and retries . . . 33
7.4.6 Mock ERP contract . . . 34

8 Traceability, Open Decisions and Next Steps **35**
8.1 Acceptance criteria of the six stories . . . 35
8.2 Open decisions . . . 36

8.3 Risks specific to this phase . . . 36
8.4 Next steps . . . 36

9 Extension: Tender and Opportunities Module **37**
9.1 Why this chapter exists . . . 37
9.2 Idea and value . . . 37
9.3 Shared design for tenders and jobs . . . 37
9.4 Roles and permissions . . . 38
9.5 Status models . . . 39
9.6 API additions . . . 39
9.7 Link to the ERP . . . 39
9.8 Where each part is built . . . 40
9.9 Open decisions and risks . . . 40
9.10 Verification performed . . . 40

A Database Schema Listing **41**

List of Figures

2.1 Layered architecture. The React interface calls versioned routers; routers call domain services; services use protected storage, MySQL and, at the bottom, the ERP. The job queue is introduced after the synchronous version works. 4

2.2 Processing pipeline from upload to ERP submission, with the human review step before the ERP. 6

2.3 Docker Compose deployment (written, not yet run on the developer's machine). 7

3.1 Entity-relationship diagram of the 16 tables. Lines show the main one-to-many relationships; the table in Section 3.5 lists every foreign key. The tables of the tender module are in Figure 9.1. 8

4.1 Document lifecycle. The filled dot is the start; double circles are end states that can still be left only by an explicit retry or reprocess action. 16

4.2 Activity view of the whole flow, including the review decision and the ERP result. 17

4.3 Upload and asynchronous processing. The user receives an identifier immediately and follows the status. 20

6.1 Navigation map. Colour shows which roles may open a screen; the role is always checked again in the backend. 26

6.2 Dashboard wireframe with synthetic values. Status is always shown as text as well as colour. 27

6.3 Upload wireframe showing a finished upload, one in progress and one refused file. 27

6.4 Review workspace wireframe (synthetic invoice). 28

7.1 Review, approval and ERP submission. 33

9.1 Tables added for the tender module. Each file carried by a listing or an application is an ordinary row of `documents`, so it uses the same upload, OCR, extraction and review path as an invoice. 38

1

Overview of Phase 2

1.1 Goal

Phase 2 answers one question: *how will the frontend, backend, database, OCR engine and ERP integration work together?* The Phase 1 requirements said what the system must do. This phase decides how it is built, in enough detail that Phase 3 onward can implement it without guessing.

1.2 User stories covered

The six stories below come from the project Kanban board. Each one is closed by a chapter of this document and, where useful, by a file that a program can check.

Story	Title	Chapter	Main deliverable
US-06	Design the system architecture	2	Layered architecture, technology decisions, module rules, deployment views.
US-07	Design the database	3	`schema.sql`: 16 tables, 146 columns, keys, checks, indexes, audit trigger.
US-08	Design the document-processing workflow	4	Stage table, state machine, validation and retry rules.
US-09	Design the REST API	5	`openapi.yaml`: 34 operations, 31 schemas, error model.
US-10	Design the user interface	6	Screen map, wireframes, state table.
US-11	Define security and ERP integration	7	Threat and control table, ERP adapter design, mapping and retry policy.

1.3 Main design decisions at a glance

Topic	Decision	Reason
Backend shape	One FastAPI application with separated modules.	Fewer moving parts for a single developer; services can be extracted later.
Database	MySQL 8 with SQLAlchemy 2 and Alembic migrations.	Project requirement; migrations make schema changes repeatable.
Processing	Asynchronous: the upload returns 202 at once.	OCR is slow; the user must not wait on a request.
OCR	Behind an `OcrEngine` interface with two implementations: Tesseract and the custom CRNN model.	Fair comparison and a working fallback.

Topic	Decision	Reason
ERP	Behind an `ErpAdapter` interface with a mock and an Odoo implementation.	The mock lets the project run without Odoo access.
Review	Mandatory before any ERP submission.	Business rule BR-1.
Identity	Short-lived bearer token; role checked in the backend on every request.	The browser is never trusted to enforce permissions.
History	Every run, correction and ERP attempt is kept.	Business rules BR-5 and BR-6; auditing.

1.4 What was checked

Verification status

Checked. `schema.sql` was loaded into a real database engine (MariaDB 10.11, which is MySQL-compatible) and nine invalid inserts were rejected as designed (Section 3.8). `openapi.yaml` passes an OpenAPI 3.0 validator. The document statuses in the API and in the database are identical. The column dictionary and endpoint catalogue in this document are *generated* from those two files.

Not checked. The schema has not been run on MySQL 8.0 itself. No endpoint other than `/health` and `/health/db` exists in code. The screens are wireframes, not a running interface. Nothing has been tried against Odoo. Timing, confidence thresholds and retry values are planning values, not measurements.

2

System Architecture

Story US-06. The chapter fixes the frontend, backend and database technologies, shows the OCR pipeline and the ERP integration in one diagram, and states the rules that keep the modules separate.

2.1 Scope and context

DocuMind AI sits between people who hold invoices and the ERP that must record them. Three kinds of people use it (operator, reviewer, administrator); Chapter 9 adds companies and applicants for the tender module. One external system receives data from it (the ERP). Everything else, including OCR, runs inside the system boundary.

2.2 Layered architecture

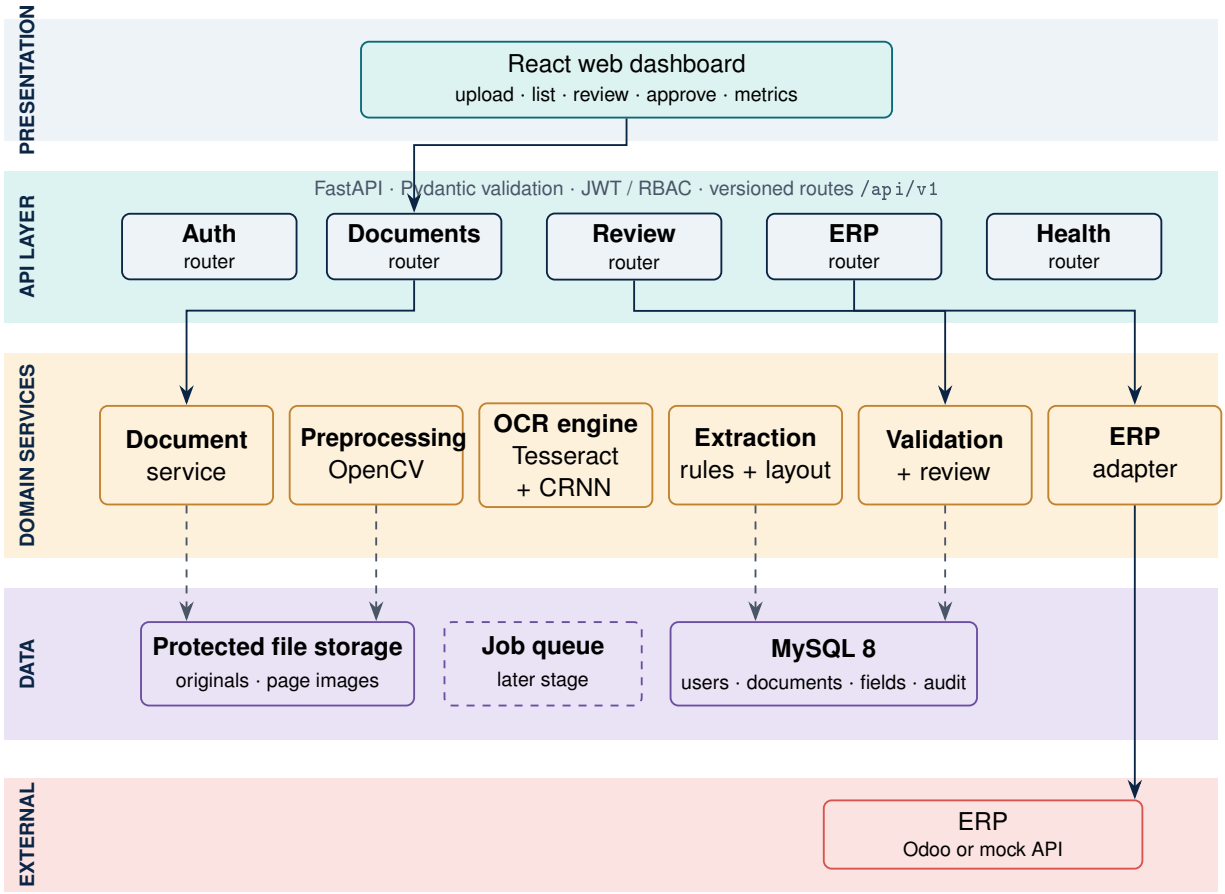


Figure 2.1: *Layered architecture. The React interface calls versioned routers; routers call domain services; services use protected storage, MySQL and, at the bottom, the ERP. The job queue is introduced after the synchronous version works.*

2.2.1 Technology decisions (acceptance criteria of US-06)

Layer	Choice	Role in the system
Web frontend	React with Vite (Node.js toolchain)	Single-page application: sign-in, upload, lists, review workspace, administration.
Backend API	FastAPI on Uvicorn	Routing, validation (Pydantic), authentication, orchestration of services.
Database	MySQL 8 (InnoDB, utf8mb4)	All structured data: users, documents, runs, fields, reviews, ERP attempts, audit.
Data access	SQLAlchemy 2 with PyMySQL; Alembic for migrations	Mapped models and versioned schema changes.
Image and PDF handling	OpenCV; PyMuPDF	Render PDF pages, deskew, denoise, binarise.
OCR	Tesseract (baseline) and a PyTorch CRNN with CTC (custom)	Text recognition behind one interface.
Storage	A private directory first; private object storage later	Original files and page images, never served directly.

Layer	Choice	Role in the system
ERP	Mock service first; Odoo through its external API	Receives approved records.
Packaging	Docker and Docker Compose; Kubernetes later	Reproducible setup; local run without Docker stays supported.

2.3 Module rules

The backend is one deployable application. To keep it maintainable the following rules apply.

1. **Direction of calls.** Router → service → repository or adapter. A router never touches the database or the file system directly, and a repository never calls a service.
2. **Interfaces at the edges.** OCR and ERP are used through interfaces (`OcrEngine`, `ErpAdapter`). A new engine or ERP is a new class, not a change to the callers.
3. **Schemas are not models.** Pydantic schemas describe the API; SQLAlchemy models describe tables. They are kept separate so a table change does not silently change the API.
4. **One place for state changes.** Document status changes happen in one service function that checks the allowed transitions (Chapter 4) and writes the audit event in the same transaction.

2.3.1 Planned backend layout

The folders `api`, `core` and `database` already exist. The others are added as their features are built.

```
backend/app/
  main.py # application and middleware (exists)
  api/ # routers: health (exists), auth, documents, review, erp, admin
  core/ # settings (exists), security, logging
  database/ # engine and session (exists), migrations
  models/ # SQLAlchemy models, one file per table group
  schemas/ # Pydantic request and response schemas
  services/ # document, workflow, validation, review, audit
  ocr/ # OcrEngine interface, tesseract engine, crnn engine, preprocessing
  erp/ # ErpAdapter interface, mock adapter, odoo adapter
  workers/ # background processing of queued documents
```

2.4 OCR pipeline and ERP integration in one view

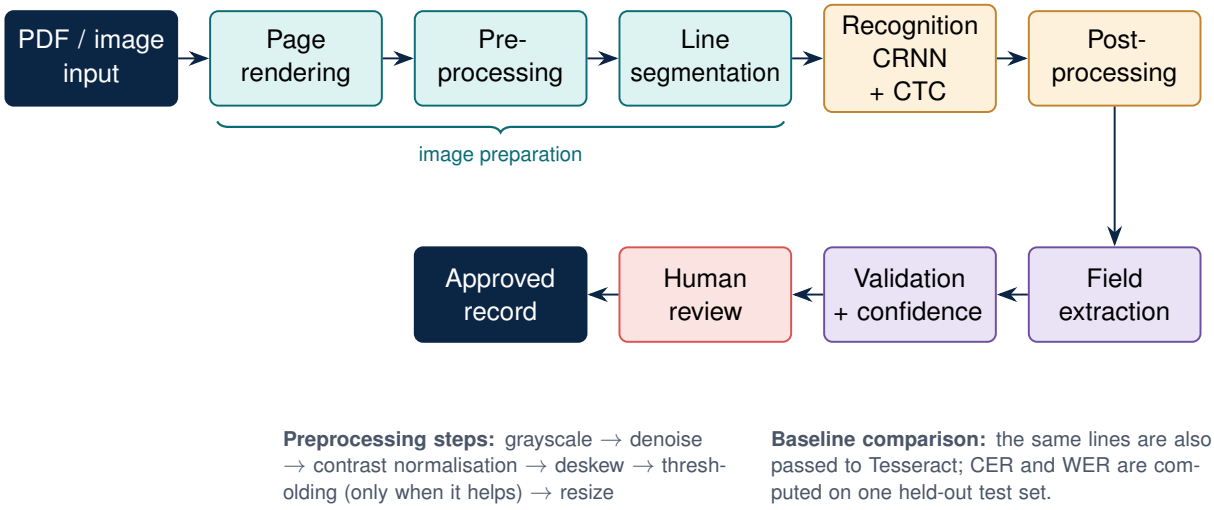


Figure 2.2: *Processing pipeline from upload to ERP submission, with the human review step before the ERP.*

2.5 Deployment views

Three ways to run the system are supported, in this order of introduction.

Mode	What runs where	Used for
Local, no Docker	Backend with Uvicorn in a virtual environment; MySQL installed on the PC; React dev server.	Daily development and debugging.
Docker Compose	Containers for the backend and MySQL; later a worker and a message broker.	Reproducible demo and clean-machine test (O10).
Kubernetes	Deployments for the API and workers; managed or stateful MySQL; secrets and config maps.	Later stage, only if time allows.

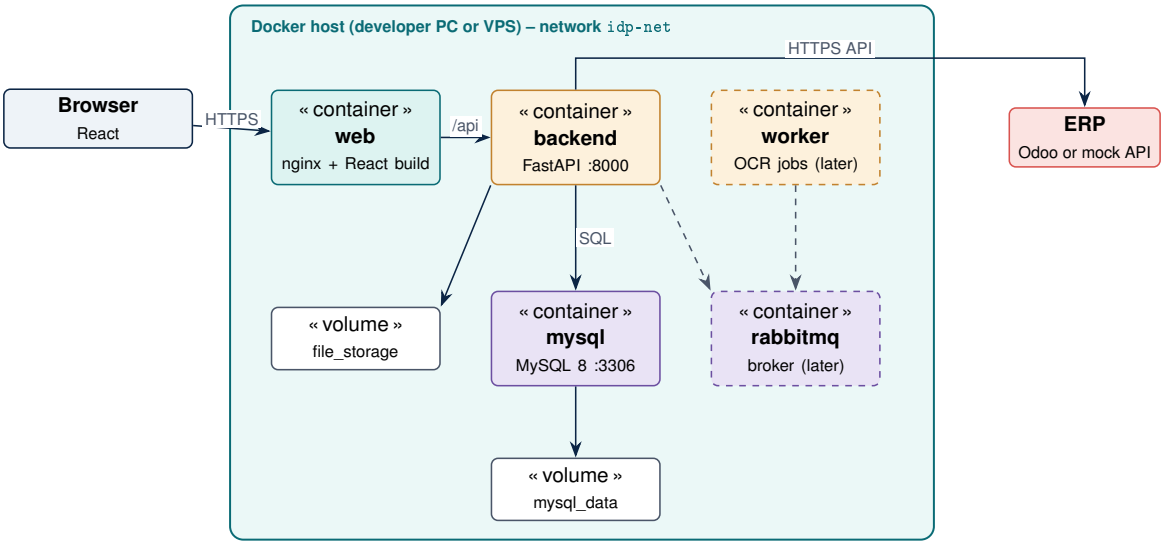


Figure 2.3: *Docker Compose deployment (written, not yet run on the developer's machine).*

2.6 Architecture decision records

ID	Decision	Alternative rejected	Consequence
A1	Modular monolith	Microservices from the start	One process to debug; boundaries must be kept by discipline.
A2	Asynchronous processing	Process during the upload request	Needs job state and a worker; the user sees status instead of waiting.
A3	Files outside the database	Store files as BLOBs	Smaller database; storage and database must be backed up together.
A4	UUID identifiers	Auto-increment integers	Identifiers do not reveal volume; slightly larger keys.
A5	Alembic migrations	SQLAlchemy `create_all`	Extra setup; every schema change is recorded and reversible.
A6	Mock ERP before Odoo	Wait for Odoo access	Work is not blocked; the adapter must be re-tested against the real ERP.
A7	Roles checked in the backend	Hide buttons in the UI only	Every router needs a role check; UI hiding is only cosmetic.
A8	Keep every processing run	Overwrite results on reprocess	More rows; history satisfies FR-21 and BR-5.

3

Database Design

Story US-07. The chapter identifies the entities, defines tables for documents, extracted fields and review decisions, fixes keys and relationships, and covers processing status and audit history.

3.1 Principles

MySQL stores structured data, not the uploaded files. Each document row holds an opaque storage key, the SHA-256 hash and file metadata. Status values, roles and decisions are restricted by `CHECK` constraints so that invalid values cannot enter even if the application has a bug. Times are stored in UTC. Nothing that must be remembered for audit is ever updated in place: runs, corrections and ERP attempts are new rows.

3.2 Entity-relationship diagram

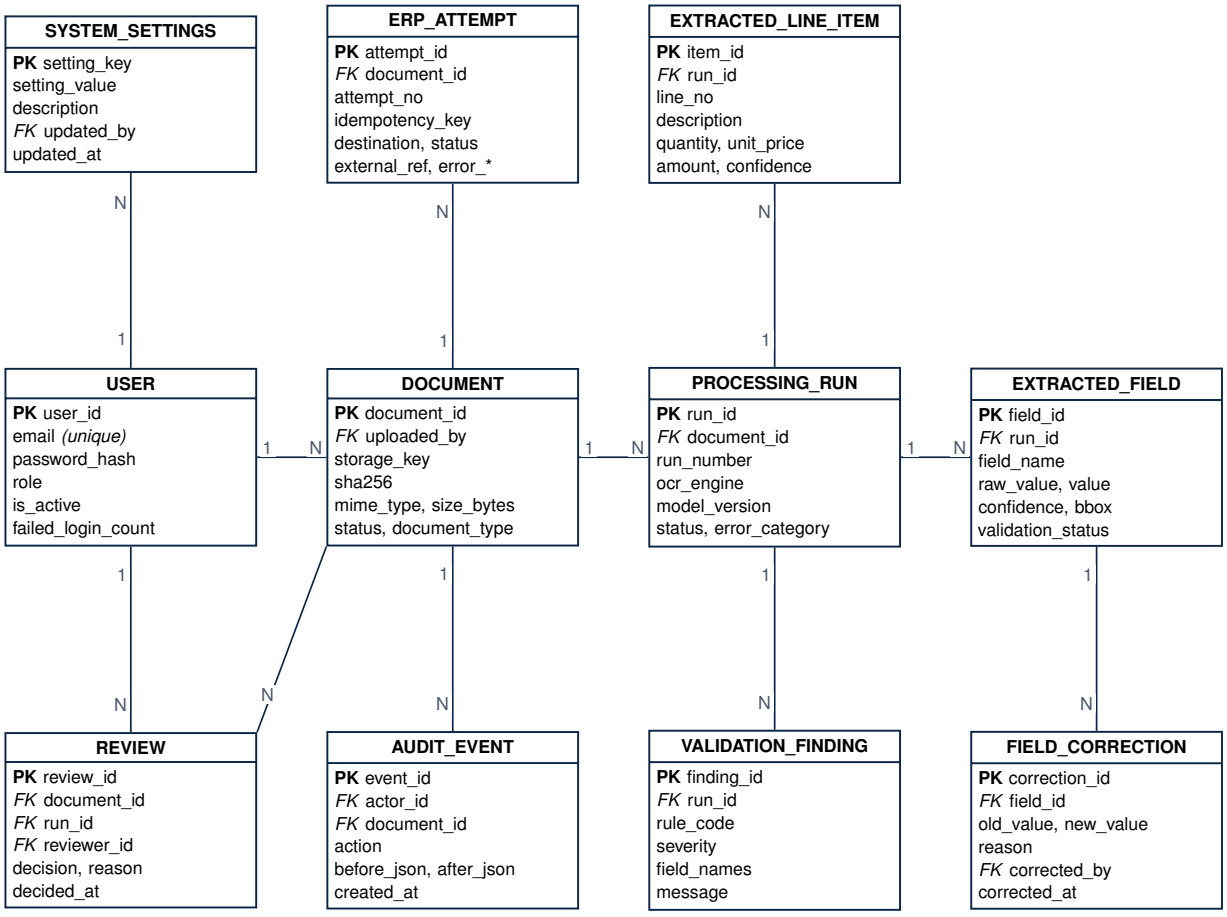


Figure 3.1: *Entity-relationship diagram of the 16 tables. Lines show the main one-to-many relationships; the table in Section 3.5 lists every foreign key. The tables of the tender module are in Figure 9.1.*

3.3 Purpose of each table

Table	Purpose
organizations	A company that publishes or answers a listing (tender module, Chapter 9).
users	Identity, role (OPERATOR, REVIEWER, ADMIN, COMPANY, APPLICANT), optional company, active flag and login-failure counter. Only a password hash is stored.
documents	One uploaded file: storage key, SHA-256, type, size, pages, language and current workflow status.
processing_runs	One processing attempt of a document, with engine, model version, preprocessing settings, status and sanitised error. Reprocessing adds a run.
extracted_fields	The header fields of one run (supplier, invoice number, dates, subtotal, tax, total, currency), with raw value, normalised value, reviewed value, confidence, page, bounding box and validation status.
extracted_line_items	Invoice table rows of one run.
validation_findings	Document-level rule results such as arithmetic mismatch, duplicate or missing field, with a severity (INFO, WARNING, BLOCKING).
reviews	The human decision (APPROVED or REJECTED) with reviewer, run, reason and time.
field_corrections	Each manual change to a field: old value, new value, reason, user and time (rule BR-5).
erp_attempts	Each submission to the ERP: attempt number, idempotency key, destination, status, ERP reference and sanitised error (rules BR-4 and BR-6).
audit_events	Append-only record of who did what and when, with before and after values.
system_settings	Administrator-editable thresholds and limits, so a change needs no redeployment (FR-17).
listings, listing_documents, applications, application_documents	Tender module: a tender, job or internship, the files it carries, the applications received and their files. Described in Chapter 9.

3.4 Column dictionary

The table below is generated from the database that was created from `schema.sql`. Types are shown as MySQL reports them; the four `json` columns are plain JSON text in MariaDB, which was used for the check.

Column	Type	Null	Key
organizations			
org_id	char(36)	no	PK
name	varchar(200)	no	UNIQUE
registration_no	varchar(60)	yes	
country	varchar(60)	yes	
contact_email	varchar(255)	yes	
is_active	tinyint(4)	no	
created_at	datetime	no	

Column	Type	Null	Key
updated_at	datetime	no	
users			
user_id	char(36)	no	PK
email	varchar(255)	no	UNIQUE
full_name	varchar(150)	no	
password_hash	varchar(255)	no	
role	varchar(20)	no	
org_id	char(36)	yes	FK → organizations
is_active	tinyint(4)	no	
failed_login_count	int	no	
locked_until	datetime	yes	
last_login_at	datetime	yes	
created_at	datetime	no	
updated_at	datetime	no	
documents			
document_id	char(36)	no	PK
uploaded_by	char(36)	no	FK → users
original_filename	varchar(255)	no	
storage_key	varchar(255)	no	UNIQUE
sha256	char(64)	no	
mime_type	varchar(100)	no	
size_bytes	bigint	no	
page_count	int	yes	
document_type	varchar(20)	no	
language	varchar(10)	yes	
status	varchar(20)	no	
retry_count	int	no	
created_at	datetime	no	
updated_at	datetime	no	
deleted_at	datetime	yes	
processing_runs			
run_id	char(36)	no	PK
document_id	char(36)	no	FK → documents
run_number	int	no	
status	varchar(20)	no	
ocr_engine	varchar(20)	no	
model_version	varchar(60)	yes	
preproc_config	json	yes	
error_category	varchar(40)	yes	
error_message	varchar(500)	yes	
started_at	datetime	no	
ended_at	datetime	yes	
extracted_fields			
field_id	char(36)	no	PK
run_id	char(36)	no	FK → processing_runs
field_name	varchar(40)	no	
raw_value	varchar(1000)	yes	
normalized_value	varchar(500)	yes	
reviewed_value	varchar(500)	yes	
confidence	decimal(4,3)	yes	

Column	Type	Null	Key
page_no	int	yes	
bbox	json	yes	
validation_status	varchar(10)	no	
validation_message	varchar(300)	yes	
extracted_line_items			
item_id	char(36)	no	PK
run_id	char(36)	no	FK → processing_runs
line_no	int	no	
description	varchar(500)	yes	
quantity	decimal(18,4)	yes	
unit_price	decimal(18,4)	yes	
amount	decimal(18,2)	yes	
confidence	decimal(4,3)	yes	
page_no	int	yes	
bbox	json	yes	
validation_findings			
finding_id	char(36)	no	PK
run_id	char(36)	no	FK → processing_runs
rule_code	varchar(40)	no	
severity	varchar(10)	no	
field_names	varchar(200)	yes	
message	varchar(500)	no	
created_at	datetime	no	
reviews			
review_id	char(36)	no	PK
document_id	char(36)	no	FK → documents
run_id	char(36)	no	FK → processing_runs
reviewer_id	char(36)	no	FK → users
decision	varchar(10)	no	
reason	varchar(500)	yes	
decided_at	datetime	no	
field_corrections			
correction_id	char(36)	no	PK
field_id	char(36)	no	FK → extracted_fields
old_value	varchar(500)	yes	
new_value	varchar(500)	yes	
reason	varchar(300)	no	
corrected_by	char(36)	no	FK → users
corrected_at	datetime	no	
erp_attempts			
attempt_id	char(36)	no	PK
document_id	char(36)	no	FK → documents
attempt_no	int	no	
idempotency_key	varchar(80)	no	
destination	varchar(20)	no	
status	varchar(10)	no	
http_status	int	yes	
external_ref	varchar(100)	yes	
error_category	varchar(40)	yes	
error_message	varchar(500)	yes	

Column	Type	Null	Key
created_at	datetime	no	
finished_at	datetime	yes	
audit_events			
event_id	bigint	no	PK
actor_id	char(36)	yes	FK → users
document_id	char(36)	yes	FK → documents
action	varchar(50)	no	
entity_type	varchar(30)	yes	
entity_id	varchar(36)	yes	
before_json	json	yes	
after_json	json	yes	
ip_address	varchar(45)	yes	
created_at	datetime	no	
system_settings			
setting_key	varchar(60)	no	PK
setting_value	varchar(255)	no	
description	varchar(255)	yes	
updated_by	char(36)	yes	FK → users
updated_at	datetime	no	
listings			
listing_id	char(36)	no	PK
org_id	char(36)	no	FK → organizations
created_by	char(36)	no	FK → users
listing_type	varchar(20)	no	
title	varchar(200)	no	
description	text	no	
requirements	text	yes	
location	varchar(120)	yes	
budget_amount	decimal(15,2)	yes	
currency	char(3)	yes	
deadline	datetime	no	
status	varchar(20)	no	
awarded_application_id	char(36)	yes	FK → applications
created_at	datetime	no	
updated_at	datetime	no	
listing_documents			
listing_id	char(36)	no	PK, FK → listings
document_id	char(36)	no	PK, FK → documents
doc_role	varchar(20)	no	
created_at	datetime	no	
applications			
application_id	char(36)	no	PK
listing_id	char(36)	no	FK → listings
applicant_id	char(36)	no	FK → users
applicant_org_id	char(36)	yes	FK → organizations
cover_note	text	yes	
status	varchar(20)	no	
submitted_at	datetime	no	
updated_at	datetime	no	
application_documents			

Column	Type	Null	Key
application_id	char(36)	no	PK, FK → applications
document_id	char(36)	no	PK, FK → documents
doc_role	varchar(20)	no	
created_at	datetime	no	

3.5 Keys, constraints and indexes

Constraint	Where	What it protects
Primary key (UUID)	Every table except audit_events, system_settings and the two link tables (which use a pair of keys).	Stable identity; audit_events uses an increasing number to keep order.
Unique	users.email; documents.storage_key; (document_id, run_number); (run_id, field_name); (run_id, line_no); (document_id, attempt_no); organizations.name; (listing_id, applicant_id).	No duplicate accounts, no two runs with the same number, one value per field per run.
Foreign keys	documents.uploaded_by; processing_runs.document_ extracted_fields.run_id; extracted_line_items.run_ validation_findings.run_i reviews.document_id, run_id, reviewer_id; field_corrections.field_i corrected_by; erp_attempts.document_id; audit_events.actor_id, document_id; system_settings.updated_b users.org_id; listings.org_id, created_by, awarded_application_id; applications.listing_id, applicant_id, applicant_org_id; the two link tables to documents.	No row can refer to something that does not exist.
Check	Role, listing type and status, application status, link-table roles, document status, document type, run status, OCR engine, validation status, severity, review decision, ERP destination and status, confidence between 0 and 1, size above 0.	Invalid values are refused by the database itself.

Constraint	Where	What it protects
Index	`documents` by status and date, by hash, and by uploader and date; `erp_attempts` by idempotency key and by status; `audit_events` by document, by actor and by action.	Review queue, duplicate-file lookup, dashboard counts and audit searches stay fast.

Duplicate invoices and double ERP submission

A duplicate *invoice* (same supplier, number, date and amount) is a business rule that produces a finding, because extracted values can be wrong or incomplete; it is not a unique constraint. A duplicate ERP *submission* is prevented by the idempotency key and by a service-layer check inside one transaction that locks the document row. The retry tests required by objective O9 must prove this.

3.6 Audit design

Audit rows are inserted by the same transaction that makes the change. Two database triggers reject `UPDATE` and `DELETE` on `audit_events`, so the trail is append-only even if application code is wrong. In deployment the application account should also lack the right to drop or alter the triggers. Actions recorded: login, upload, status change, field correction, approval, rejection, ERP submission and setting change.

3.7 Status and history

The `documents.status` column holds the nine states of Figure 4.1 (Chapter 4). Processing history lives in `processing_runs`; a reprocess adds a run with the next `run_number` and keeps all earlier runs and their fields. The latest run is the one with the highest number.

3.8 Verification performed

`schema.sql` was loaded into an empty database on MariaDB 10.11, which accepted all 16 tables and the starting settings. Sample rows were inserted, then nine invalid operations were tried on the final version of the schema, which includes the tender module (Chapter 9). Each one was refused.

No.	Invalid operation tried	Result
1	Second user with an existing e-mail	Refused: unique key.
2	User with role `BOSS`	Refused: role check.
3	Document with status `DONE`	Refused: status check.
4	Document uploaded by a user that does not exist	Refused: foreign key.
5	Two `total_amount` fields in the same run	Refused: unique key.
6	Confidence of 1.5	Refused: range check.
7	Two runs with number 1 for the same document	Refused: unique key.
8	`UPDATE` of an audit event	Refused by trigger.
9	`DELETE` of an audit event	Refused by trigger; the row was still there afterwards.

These tests show the schema does what it was designed to do on one engine. They are not a substitute for running the same file on MySQL 8.0 and then running the migrations from Phase 3.

3.9 From design to migrations

Phase 3 turns this file into SQLAlchemy models and one initial Alembic revision. Three points need care: the `CHECK` constraints and the two triggers are not generated automatically from models and must be added with explicit SQL in the revision; the `JSON` columns must map to the JSON type; and the model and the revision must be compared with the live schema after the first run so they cannot drift apart.

3.10 Retention and privacy

Invoices can contain personal and financial data. The `deleted_at` column supports removal of a document from normal use, and a retention period must be chosen before real data is used (open decision, Section 8.2). Demonstrations use synthetic or authorised documents only. Error messages stored in the database are sanitised and never contain file paths or stack traces.

4

Document-Processing Workflow

Story US-08. The chapter defines upload and file validation, preprocessing and OCR, extraction and confidence scoring, human review of uncertain results, and the path of an approved document to the ERP.

4.1 Lifecycle

Every document is in exactly one state. The backend refuses any change that is not an arrow in Figure 4.1; for example an unreviewed document can never jump to `ERP_SENT`.

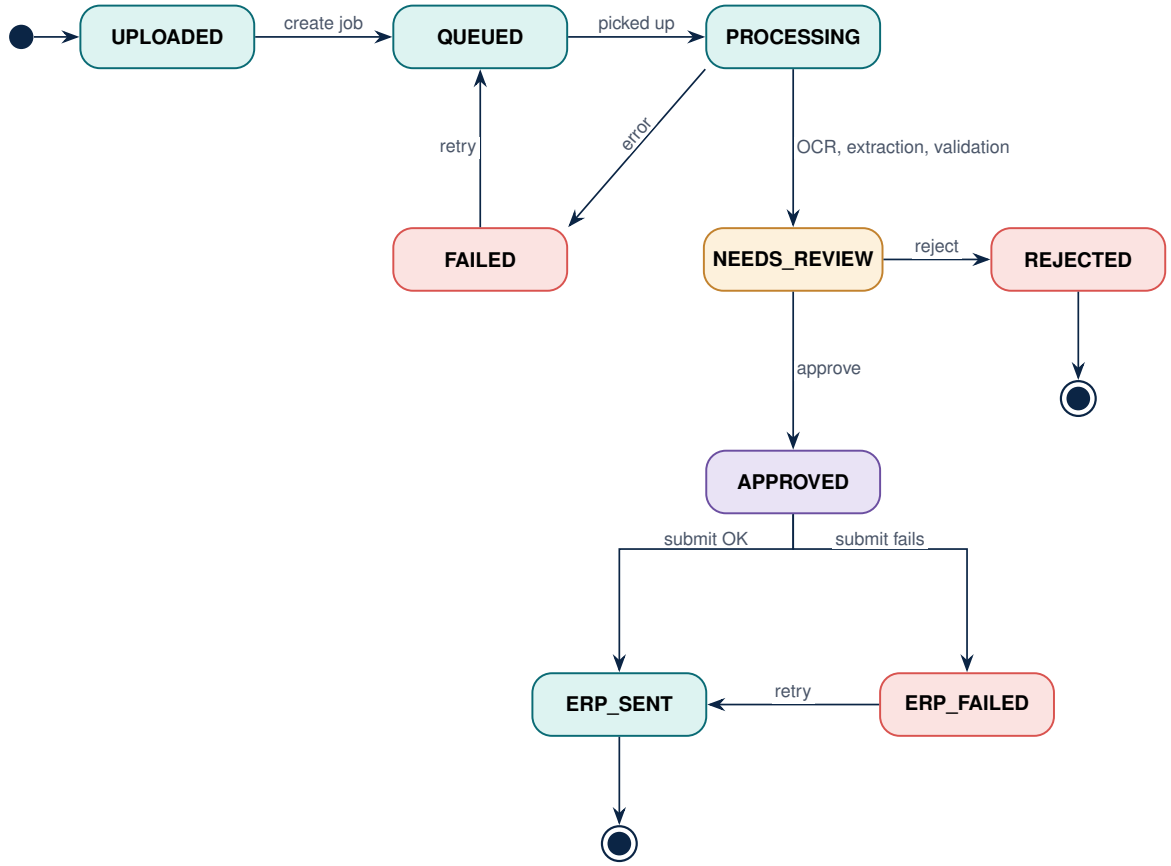


Figure 4.1: *Document lifecycle. The filled dot is the start; double circles are end states that can still be left only by an explicit retry or reprocess action.*

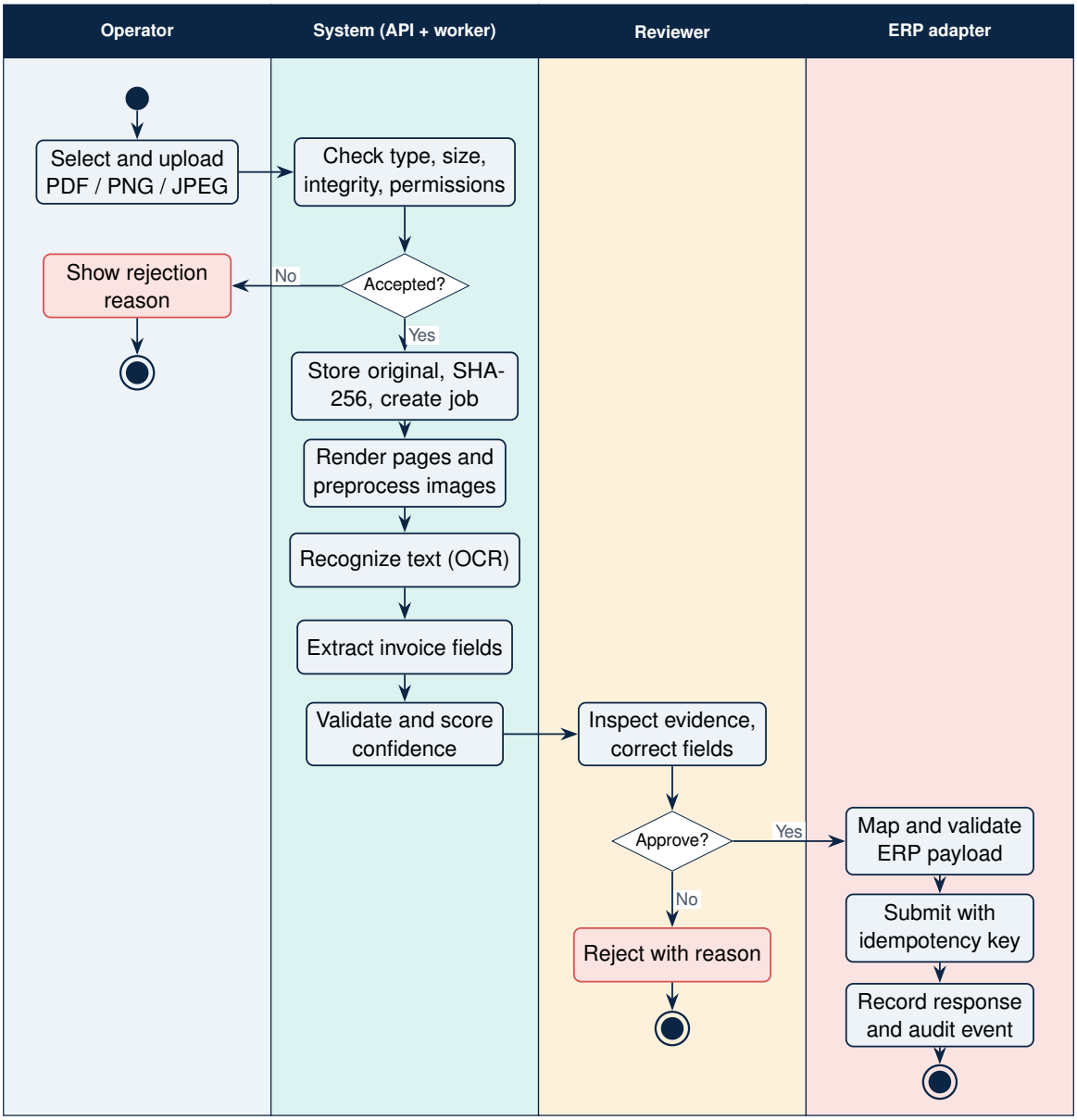


Figure 4.2: *Activity view of the whole flow, including the review decision and the ERP result.*

4.2 Stage by stage

Stage	What happens	Writes	Status after	On failure
1. Receive and validate	Check type, content, size, pages and integrity (Section 4.3).	Nothing yet if refused.	—	Refuse with 400, 413 or 415.
2. Store	Save the original in protected storage under an opaque key; compute SHA-256.	`documents` row; audit event UPLOAD.	UPLOADED	Delete the partial file; return 500.
3. Queue	Mark ready for the worker; return 202 with the document ID.	Status.	QUEUED	Stays QUEUED; the worker picks it up later.
4. Preprocess	Render PDF pages to images; deskew, denoise, normalise contrast; segment lines.	`processing_runs` row; page images in storage.	PROCESSING	Category UNREADABLE or TIMEOUT.

Stage	What happens	Writes	Status after	On failure
5. Recognise	Run the selected OCR engine; keep text, line boxes and confidence.	Run engine and model version.	PROCESSING	Category OCR_ERROR.
6. Extract	Map recognised text to the invoice fields and line items; keep page and box.	`extracted_fields`, `extracted_line_items`.	PROCESSING	Fields left empty and flagged, not invented.
7. Validate	Apply the rules of Section 4.4; compute findings and a validation status per field.	`validation_findings`.	PROCESSING	Rule error is logged; record still goes to review.
8. Route	Always send to human review in the prototype.	Run status SUCCEEDED.	NEEDS_REVIE	Unrecoverable error: FAILED.
9. Review	Reviewer corrects fields with a reason, then approves or rejects.	`field_corrections`, `reviews`, audit.	APPROVED or REJECTED	—
10. Submit	Build the ERP payload; send with the idempotency key; record the result.	`erp_attempts`, audit.	ERP_SENT or ERP_FAILED	Retry is safe (Chapter 7).

4.3 File validation at upload

Check	Rule	Response
Empty file	Size must be above zero.	400 `VALIDATION_ERROR`
File type	Extension is `.pdf`, `.png`, `.jpg` or `.jpeg`, and the first bytes of the content match PDF, PNG or JPEG. The client-supplied MIME type is not trusted on its own.	415 `UNSUPPORTED_MEDIA_TYPE`
Size	At most `max_upload_bytes` (10 MB at the start).	413 `FILE_TOO_LARGE`
Pages	PDF has at most `max_pdf_pages` pages (10 at the start).	400 `VALIDATION_ERROR`
Integrity	The file opens with the PDF or image library.	400 `VALIDATION_ERROR`
Duplicate file	Same SHA-256 as an earlier document: accepted, then reported as an information finding so a person decides.	202 with a warning

4.4 Validation rules

Rule code	Check	Severity	Setting
REQUIRED_FIELD	Supplier, invoice number, invoice date, subtotal, tax, total and currency are present and parse.	BLOCKING	—
DATE	Dates parse; due date is not before invoice date; date is in a plausible range.	WARNING	—
ARITHMETIC	Subtotal plus tax equals total within the tolerance. Never corrected automatically.	WARNING	`arithmetic_tolerance`
CURRENCY	Currency code or symbol is known and consistent across fields.	WARNING	—

Rule code	Check	Severity	Setting
DUPLICATE	Same supplier, invoice number, date and amount as an earlier approved or sent document.	WARNING	—
ERP_SCHEMA	The approved values satisfy the ERP payload schema.	BLOCKING	—

A BLOCKING finding prevents approval until the reviewer corrects the cause. A WARNING does not block, but the approval event stores which warnings were open, so the audit trail shows the reviewer saw them.

4.5 Confidence and review routing

Confidence is a signal for the reviewer, not proof of correctness. The two thresholds live in `system_settings` and the starting values (0.90 and 0.70) are **provisional**. They must be chosen on validation data after the OCR baseline exists, never on the test set.

Condition on a field	Shown to the reviewer as	Reviewer action
Confidence at or above `confidence_accept` and no failed rule	Accepted (green, with a text label).	May confirm without changes.
Between `confidence_review` and `confidence_accept`	Highlighted (amber, REVIEW).	Should check against the source.
Below `confidence_review`, or a failed rule	Needs attention (red, REVIEW or FAIL).	Must confirm or correct before approval.

In the prototype every document goes to review, whatever the confidence (rule BR-1). A later version could skip review for very high confidence, but only after measured error rates justify it.

4.6 Failure and retry

Error category	Kind	Behaviour (values provisional)
UNREADABLE	Permanent	Run FAILED, document FAILED. A person uploads a better file or requests reprocessing.
OCR_ERROR	Possibly temporary	Up to two automatic retries, then FAILED.
TIMEOUT	Temporary	Up to two automatic retries with a longer limit, then FAILED.
STORAGE_ERROR	Temporary	Retry; if it keeps failing, FAILED and an alert in the log.
INTERNAL	Unknown	FAILED; details only in the server log, not in the response.

`documents.retry_count` records automatic retries. A manual reprocess from FAILED or REJECTED creates a new run and returns the document to QUEUED; the earlier runs stay.

4.7 Upload sequence

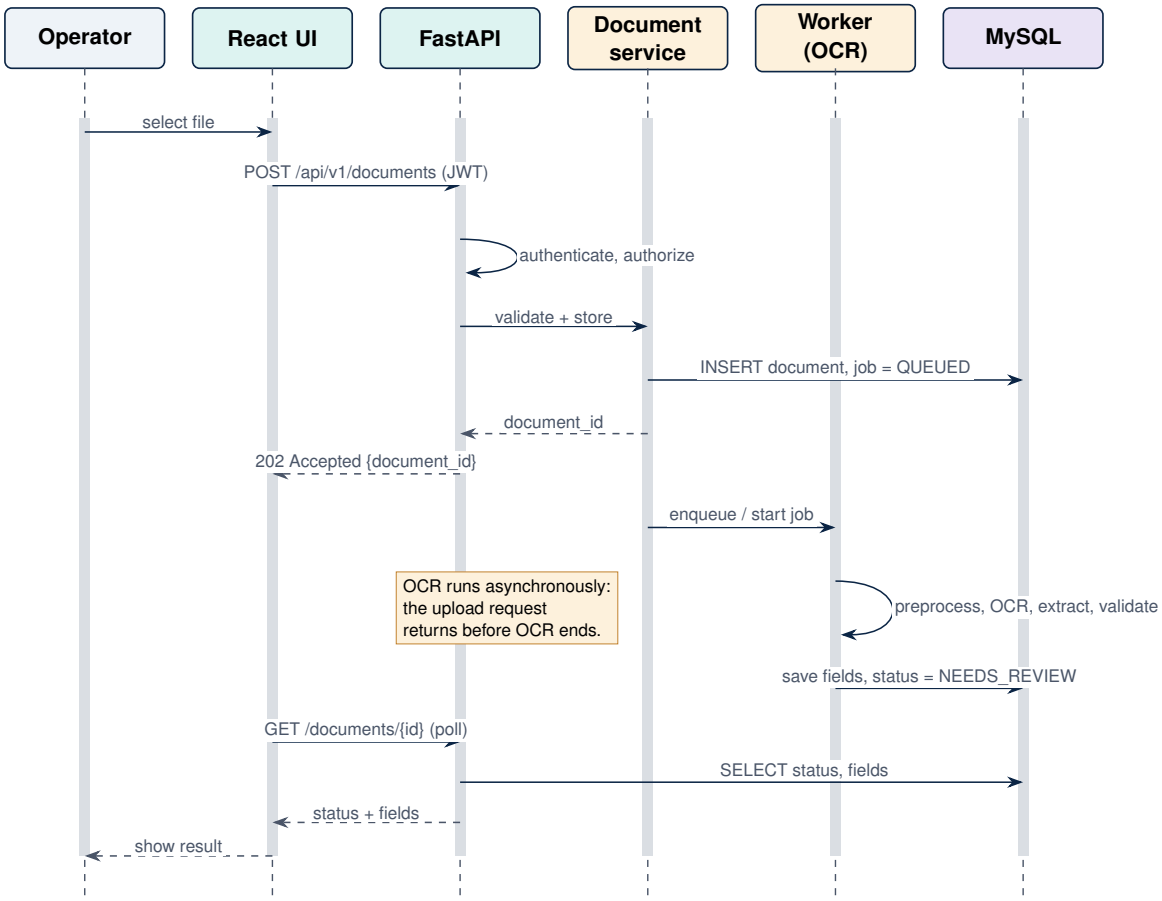


Figure 4.3: *Upload and asynchronous processing. The user receives an identifier immediately and follows the status.*

5

REST API Design

Story US-09. The chapter defines endpoints for uploading and retrieving documents, for extracted fields and review decisions, the response formats and error handling, and the authentication and authorisation rules. The complete contract is the file `openapi.yaml`.

5.1 Conventions

- Base path `/api/v1`; JSON bodies; `multipart/form-data` for uploads.
- Names in `snake_case`; identifiers are UUID strings; times are ISO 8601 in UTC.
- Lists are paged with `page` (from 1) and `page_size` (default 20, at most 100) and return `items`, `page`, `page_size` and `total`.
- Clients must ignore unknown response fields, so fields can be added without a new version. A breaking change would become `/api/v2`.
- Every response carries a request identifier that also appears in the server log.

5.2 Authentication and authorisation

A user signs in with e-mail and password at `POST /auth/login` and receives a short-lived bearer token (the contract example uses 1800 seconds, a provisional value). Every other endpoint, except the two health checks, requires the token. The backend looks up the user and role on each request and refuses with `401` (no or invalid token) or `403` (role not allowed). Repeated failed logins lock the account for a short time and the login endpoint returns `429` when rate-limited.

Function	Operator	Reviewer	Admin
Sign in, read own profile	yes	yes	yes
Upload; list and view own documents	yes	yes	yes
List and view *all* documents; dashboard of all documents	—	yes	yes
Request reprocessing	own	yes	yes
Read extracted fields; view source file and page images	—	yes	yes
Correct fields; approve; reject	—	yes	yes
Submit to ERP; view integration history; export	—	yes	yes
View a document's audit trail	—	yes	yes
Manage users; change settings	—	—	yes

A document that a user may not see is reported as `404`, not `403`, so the API does not confirm that it exists. The roles Company and Applicant of the tender module have their own permission table in Chapter 9.

5.3 Endpoint catalogue

The table is generated from `openapi.yaml`. *Roles* abbreviates Operator (Op), Reviewer (Rev), Administrator (Adm), Company (Co) and Applicant (App). The ten operations of the tender module (Chapter 9) are included.

Method	Path (after `/api/v1`)	Roles	Purpose	Status codes
GET	`/health`	public	Liveness check *(exists)*	200
GET	`/health/db`	public	Database readiness check *(exists)*	200, 503
POST	`/auth/login`	public	Sign in and receive an access token	200, 401, 429
GET	`/auth/me`	Op, Rev, Adm	Current user profile	200, 401
POST	`/documents`	Op, Rev, Adm	Upload a document and create a processing job	202, 400, 401, 413, 415
GET	`/documents`	Op, Rev, Adm	List and filter documents	200, 401
GET	`/documents/{document_id}`	Op, Rev, Adm	Document metadata, status and processing summary	200, 401, 404
GET	`/documents/{document_id}/result`	Rev, Adm	Extracted fields, line items and validation findings of the latest run	200, 403, 404, 409
GET	`/documents/{document_id}/file`	Rev, Adm	Download the original file after an authorisation check	200, 403, 404
GET	`/documents/{document_id}/pages/{page_no}/image`	Rev, Adm	Rendered page image for the review screen	200, 403, 404
PATCH	`/documents/{document_id}/fields`	Rev, Adm	Save reviewer corrections (reason required, original value kept)	200, 400, 403, 404, 409
POST	`/documents/{document_id}/approve`	Rev, Adm	Approve the record (blocked while a BLOCKING finding is open)	200, 403, 404, 409
POST	`/documents/{document_id}/reject`	Rev, Adm	Reject the record with a reason	200, 400, 403, 409
POST	`/documents/{document_id}/reprocess`	Op, Rev, Adm	Start a new processing run for a FAILED or REJECTED document	202, 403, 409
POST	`/documents/{document_id}/submit`	Rev, Adm	Send an APPROVED record to the ERP adapter (idempotent)	200, 403, 409, 502
GET	`/documents/{document_id}/erp-attempts`	Rev, Adm	Integration history of one document	200, 403, 404
GET	`/documents/{document_id}/audit`	Rev, Adm	Audit trail of one document	200, 403, 404
GET	`/dashboard/summary`	Op, Rev, Adm	Counts by status, review queue size and recent failures	200
GET	`/exports/documents`	Rev, Adm	Export approved records as JSON or CSV	200, 403
GET	`/users`	Adm	List users	200, 403
POST	`/users`	Adm	Create a user	201, 400, 403, 409
PATCH	`/users/{user_id}`	Adm	Change role or deactivate a user	200, 403, 404
GET	`/settings`	Adm	Read thresholds and validation settings	200, 403
PUT	`/settings/{setting_key}`	Adm	Change one setting (audited)	200, 400, 403, 404

Method	Path (after /api/v1)	Roles	Purpose	Status codes
GET	/listings	Op, Rev, Adm, Co, App	Browse listings (tenders, jobs, internships)	200, 401
POST	/listings	Co, Adm	Create a listing as a draft	201, 400, 403
GET	/listings/{listing_id}	Op, Rev, Adm, Co, App	One listing	200, 404
PATCH	/listings/{listing_id}	Co, Adm	Edit a draft, publish it (OPEN), close or cancel it	200, 403, 404, 409
POST	/listings/{listing_id}/applications	Co, App	Apply to an OPEN listing before its deadline (one application per person)	201, 400, 403, 404, 409
GET	/listings/{listing_id}/applications	Co, Adm	Applications received for a listing (publishing company only)	200, 403, 404
PATCH	/applications/{application_id}	Co, App, Adm	Withdraw (applicant) or change status (publishing company)	200, 403, 404, 409
GET	/applications/mine	Co, App	My own applications and their status	200, 401
GET	/listings/{listing_id}/comparison	Co, Adm	Side-by-side comparison of the offers of a tender (extracted price, delivery time, validity)	200, 403, 404, 409
POST	/listings/{listing_id}/award	Co, Adm	Award a tender to one application and trigger the purchase order to the ERP	200, 403, 404, 409

5.4 Examples

5.4.1 Sign in

```
POST /api/v1/auth/login
{ "email": "reviewer@example.org", "password": "********" }

200 OK
{ "access_token": "<jwt>", "token_type": "bearer", "expires_in": 1800,
  "user": { "user_id": "7c1f...", "email": "reviewer@example.org",
            "full_name": "A. Reviewer", "role": "REVIEWER", "is_active": true } }
```

5.4.2 Upload

```
POST /api/v1/documents (multipart, field "file")

202 Accepted
{ "document_id": "b3a9...", "original_filename": "INV-2026-001.pdf",
  "status": "QUEUED", "created_at": "2026-10-16T08:12:03Z" }
```

5.4.3 Extraction result

```
GET /api/v1/documents/b3a9.../result

200 OK
{ "document_id": "b3a9...", "status": "NEEDS_REVIEW", "run_id": "55d0...",
  "review_required": true,
  "fields": [
```

```
    { "name": "invoice_number", "raw_value": "INV-2026-001", "value": "INV-2026-001",
      "confidence": 0.91, "validation": "PASS", "message": null, "page": 1,
      "bbox": { "x": 0.61, "y": 0.08, "w": 0.22, "h": 0.03 } },
    { "name": "total_amount", "raw_value": "125 000", "value": "125000.00",
      "confidence": 0.68, "validation": "REVIEW",
      "message": "Subtotal + tax does not equal total", "page": 1, "bbox": null } ],
  "line_items": [],
  "findings": [
    { "rule_code": "ARITHMETIC", "severity": "WARNING",
      "message": "Subtotal + tax differs from total by 3.00",
      "field_names": ["subtotal", "tax_amount", "total_amount"] } ] }
```

The values are synthetic examples that show the format; they are not output of an implemented endpoint.

5.4.4 Correction

```
PATCH /api/v1/documents/b3a9.../fields
{ "corrections": [
    { "name": "total_amount", "new_value": "125003.00",
      "reason": "Digit misread; total printed as 125 003" } ] }
```

The service stores the old value, the new value, the reason, the user and the time in `field_corrections`, updates `reviewed_value`, re-runs the validation rules for the document and returns the new result.

5.5 Error model

Every error has the same shape, so the frontend needs one error handler.

```
409 Conflict
{ "error": { "code": "INVALID_STATE",
             "message": "Document must be APPROVED before submission",
             "details": [], "request_id": "req_9f2c" } }
```

HTTP	error.code	When
400	VALIDATION_ERROR	Malformed body, missing field, bad value, corrupt file, too many pages. `details` lists each field and issue.
401	UNAUTHENTICATED	Missing, invalid or expired token; wrong password.
403	FORBIDDEN	The role may not perform the action.
404	NOT_FOUND	No such resource, or the user may not see it.
409	INVALID_STATE	The status does not allow the action (for example approve while PROCESSING), or a blocking finding is open, or a user already exists.
413	FILE_TOO_LARGE	Upload above the size limit.
415	UNSUPPORTED_MEDIA_TY	File type not accepted.
429	RATE_LIMITED	Too many sign-in attempts.
502	ERP_UNAVAILABLE	The ERP rejected the record or could not be reached. The attempt is stored and retry is safe.
500	INTERNAL_ERROR	Unexpected error. The message is generic; details are only in the server log.

Responses never contain stack traces, file-system paths, credentials or the internals of the ERP.

5.6 Idempotency and concurrency

- `POST /documents/{id}/submit` is safe to repeat. The key sent to the ERP is derived from the document identifier, so every retry uses the same key. If the document is already `ERP_SENT`, the call returns the successful attempt and sends nothing.
- Approve, reject and submit run in one transaction that locks the document row. Two reviewers acting at the same moment cannot both succeed: the second receives `409`.
- The double-click problem in the interface is handled twice: the button is disabled after the first click, and the backend is idempotent anyway.

5.7 How the contract will be used

The contract is the shared reference for the React client and the FastAPI routers. When the routers are implemented, the schema FastAPI generates itself is compared with `openapi.yaml`, and differences are resolved deliberately, either by fixing the code or by revising the contract. A contract test for each endpoint is part of the Phase 3 and later work.

6

User Interface Design

Story US-10. The chapter sketches the dashboard, the upload page and the extraction-results and review screens, and defines the loading, success and error states.

6.1 Users and tasks

User	Main tasks on screen
Operator	Upload invoices, see whether each one was accepted, follow its status, request reprocessing of a failed file.
Reviewer	Work through the review queue: compare the source page with the extracted values, correct, approve or reject, then submit to the ERP and check the result.
Administrator	Create users and set roles, adjust thresholds and limits, read the audit trail.

6.2 Screen map

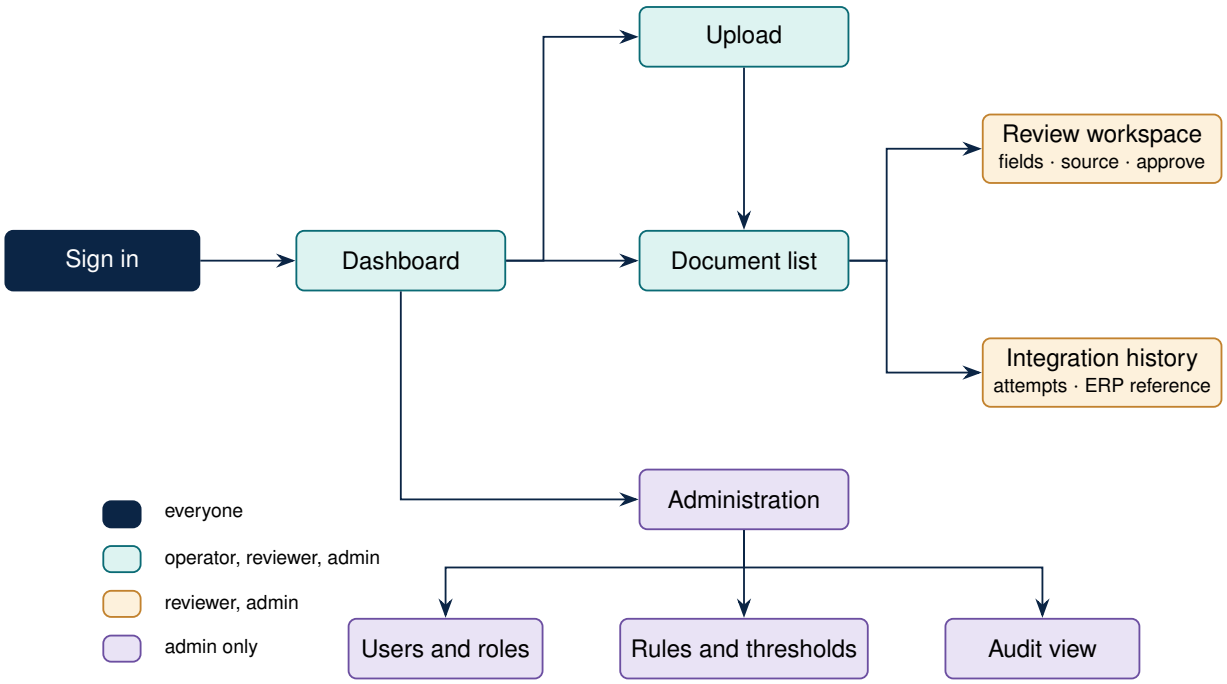


Figure 6.1: *Navigation map. Colour shows which roles may open a screen; the role is always checked again in the backend.*

6.3 Dashboard

The dashboard answers three questions at a glance: how much work is waiting, what failed, and what needs a person. The review queue is the main element and lists the oldest items first.

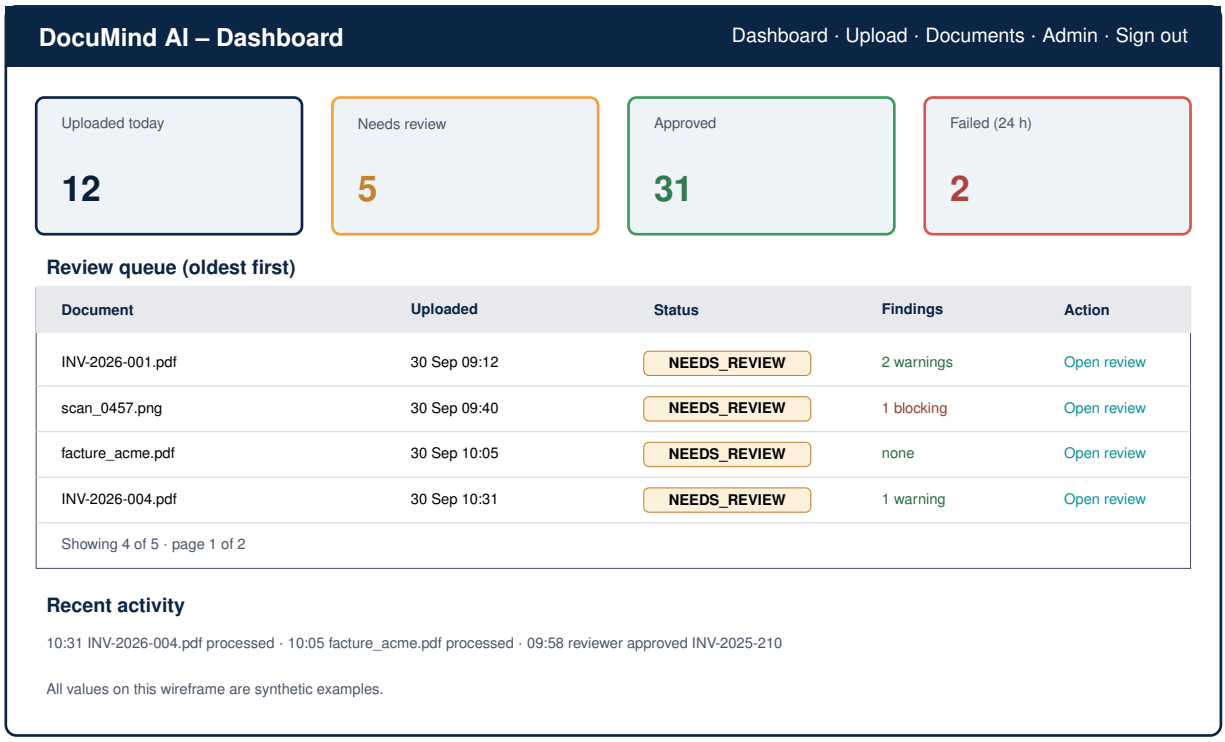


Figure 6.2: *Dashboard wireframe with synthetic values. Status is always shown as text as well as colour.*

6.4 Upload page

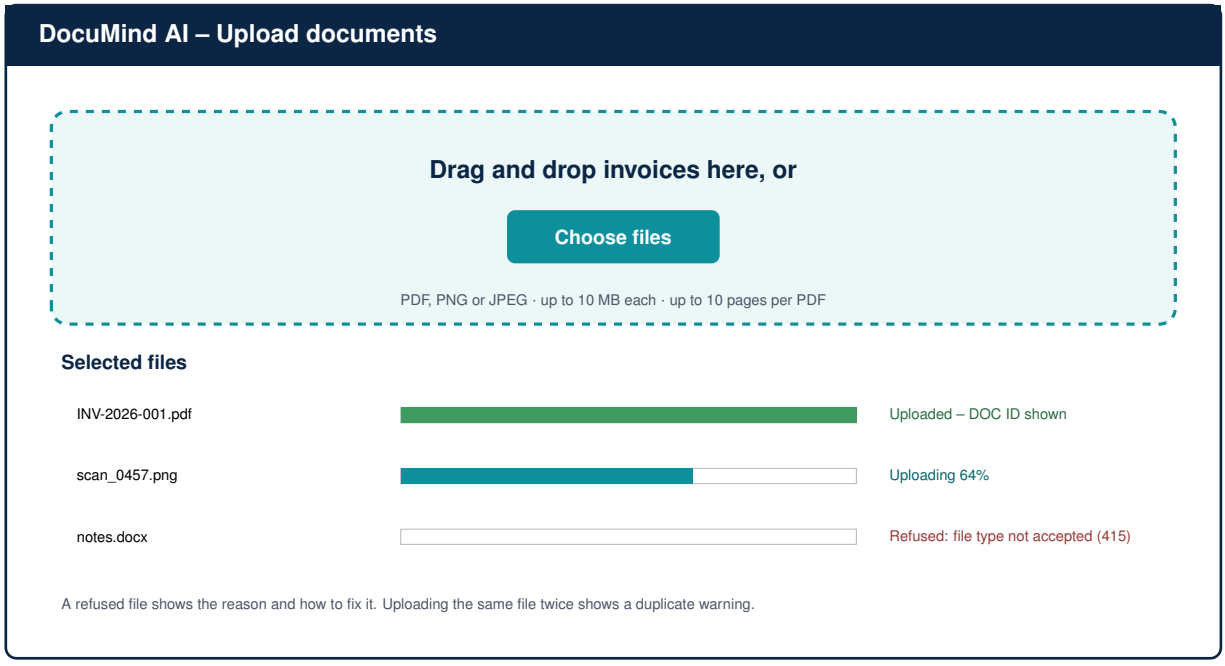


Figure 6.3: *Upload wireframe showing a finished upload, one in progress and one refused file.*

The accepted formats and limits are shown before the user chooses a file, so refusals are rare, and a refused file states the reason and what to do.

6.5 Extraction results and review

The review workspace puts the source page on the left and the editable fields on the right. Selecting a field highlights its region on the page (FR-22). A field below the review threshold or with a failed rule is marked with the word REVIEW as well as a colour. Approve is disabled while a BLOCKING finding is open and explains why. Rejecting asks for a reason.

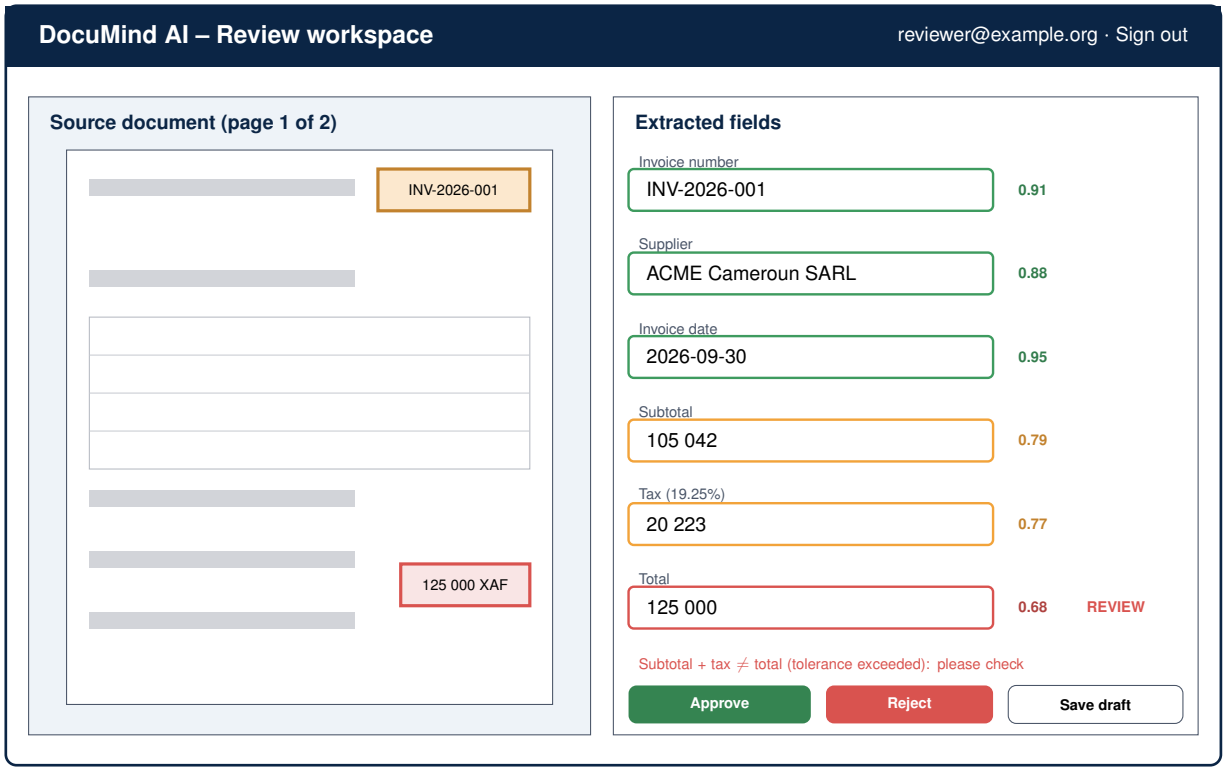


Figure 6.4: *Review workspace wireframe (synthetic invoice).*

6.6 Other screens

Screen	Content
Sign in	E-mail and password; clear message for wrong credentials and for a locked account.
Document list	Search by ID, file name or invoice number; filter by status, type and date; paged table; link to detail.
Integration history	Attempts for one document: time, status, ERP reference, sanitised error, retry button when allowed.
Users and roles	Table of users, create form, role and active switch.
Rules and thresholds	The settings from `system_settings` with description and last change; each change is audited.
Audit view	Filter by document, user and action; read-only.

6.7 Loading, success, empty and error states

Screen	Loading	Empty	Success	Error
Sign in	Button shows a spinner and is disabled.	—	Go to dashboard.	Message under the form; no hint which of e-mail or password was wrong.
Dashboard	Placeholder tiles.	“Nothing waiting” message.	Counts and queue.	Banner with retry; page stays usable.
Upload	Progress bar per file.	Drop zone with instructions.	Document ID and status shown.	Reason and fix per file (413, 415, 400).
Document list	Skeleton rows.	“No documents match” with a clear-filters link.	Rows.	Banner with retry.
Review	Source and fields load independently.	Document not ready yet: show current status.	Toast “Approved”.	Field-level messages; conflict (409) explains the current state.
Integration history	Skeleton rows.	“Not submitted yet”.	Reference shown.	Sanitised error and retry.
Administration	Spinner on save.	No settings changed yet.	Toast “Saved”.	Validation message next to the field.

6.8 Frontend structure and usability

The React application is built with Vite. It keeps one API client module that adds the token, parses the error format of Chapter 5 and exposes typed functions; screens never call the network directly. Server data is requested through a data-fetching layer (a library such as TanStack Query is proposed, to be confirmed in Phase 6). The application holds no database or ERP credentials. The token is kept in memory so that it is lost when the tab closes; whether to use a refresh mechanism is an open decision (Section 8.2).

Usability checks planned for the prototype: a keyboard-only pass through sign-in, upload and review; status never conveyed by colour alone; text contrast checked; and a short task test in which a reviewer finds and fixes a planted wrong value, as required by objective O7.

7

Security and ERP Integration

Story US-11. The chapter defines access permissions and credential management, file validation and data protection, the target Odoo version and available API, and validation and error handling before ERP submission.

7.1 Threats and controls

Threat	Control	Enforced in	How it will be tested
Unauthorised access to documents	Bearer token, role check on every route, ownership check for operators, `404` for hidden items.	Backend	Test each role against each endpoint.
Password guessing	Password hashing, lockout counter, rate limit on login.	Backend	Repeated failed logins.
Malicious or corrupt upload	Type, content, size and page checks; files stored outside the web root under opaque keys; original filename never used as a path.	Backend, storage	Wrong-type, oversized and renamed files.
Path traversal or file disclosure	No path in any response; file served only after an authorisation check.	Backend	Request files without or with the wrong role.
Injection	Parameterised queries through the ORM; Pydantic validation of all input.	Backend	Hostile strings in every text field.
Tampering with history	Append-only audit table with triggers; corrections stored, never overwritten.	Database	Update and delete attempts (done, Section 3.8).
Secret leakage	Secrets only in environment variables or a secret store; `.env` excluded from Git; tokens and passwords redacted from logs.	Deployment, backend	Repository and log review.
Cross-site calls	CORS limited to known origins (already implemented and tested).	Backend	Allowed and blocked origin tests.
Duplicate or lost ERP record	Idempotency key; transaction with row lock; stored attempts.	Backend, database	Retry and failure-injection tests.
Data exposure through logs or errors	Generic error messages; sanitised error text stored.	Backend	Force errors and inspect responses and logs.

7.2 Credentials and configuration

- Database password, token signing key and ERP credentials are read from environment variables. The repository contains only `.env.example` with placeholders.
- The ERP account is a dedicated integration user with only the rights needed to create vendor bills, and an API key rather than a personal password.

- Credentials are different for development, demonstration and any later deployment, and can be rotated without code changes.
- Passwords are stored with a slow, salted hash (Argon2id or bcrypt); the password is never logged.

7.3 Data protection

Invoices contain business and sometimes personal data. Access follows the role table of Chapter 5. Stored files are not publicly reachable. Transport uses HTTPS outside local development. Documents are not sent to external OCR or AI services. Backups of the database and the file store are taken together, and restoration is tested once. A retention period and a deletion procedure must be decided before real data is used.

7.4 ERP integration

7.4.1 Target and options

The target ERP is Odoo, reached through its external API and never by writing into its database. Odoo changes its external API between versions, so the version decides the implementation.

Option	Reported situation	Use in this project
Mock ERP	Small service written for the project; same contract as the adapter.	First target; used for all automated tests.
XML-RPC / JSON-RPC	Reported as the most compatible choice for Odoo 14 to 18 and scheduled for removal in Odoo 20.	Implement if the available Odoo is version 18 or older.
JSON-2 API (bearer key, `/json/2/{model}/{method}`)	Reported as introduced in Odoo 19 as the replacement.	Implement if the available Odoo is version 19 or newer.

Verify before implementing

The statements in this table come from a third-party summary of the Odoo documentation (Apideck, "How to integrate with the Odoo API"), not from the official page for a specific version, which could not be read during this phase. The exact endpoints, authentication and removal dates must be confirmed in the official documentation of the Odoo version that is actually available. This is open decision P2-D1.

7.4.2 Adapter interface

The rest of the system sees only this interface. A mock, an XML-RPC and a JSON-2 adapter can implement it.

```
class ErpAdapter(Protocol):
    name: str # "MOCK" or "ODOO"
    def check_payload(self, record: ApprovedRecord) -> list[Finding]: ...
    def submit(self, record: ApprovedRecord, idempotency_key: str) -> ErpResult: ...

class ErpResult:
    ok: bool
    external_ref: str | None # id of the created vendor bill
    error_category: str | None # AUTH, VALIDATION, TIMEOUT, SERVER, DUPLICATE
    message: str | None # sanitised
    http_status: int | None
```

7.4.3 Field mapping

The Phase 1 field list maps to a vendor bill. The Odoo names are candidates for a vendor bill (`account.move` with the vendor-bill type) and must be checked against the chosen version.

Extracted field	Candidate Odoo field	Note
Supplier name, tax ID	Partner (`partner_id`)	Find by tax ID or name; creating a new partner automatically is not done. If none matches, the reviewer chooses or the submission is blocked (ERP_SCHEMA).
Invoice number	Vendor reference (`ref`)	Also used for the duplicate check on the ERP side.
Invoice date	Invoice date	ISO format.
Due date	Due date	Optional.
PO reference	Origin	Optional.
Line items	Invoice lines	Quantity, unit price and description per line; if no lines were extracted, one line with the subtotal is proposed and the reviewer confirms.
Subtotal, tax, total	Computed by the ERP from lines and taxes	Compared with the extracted values; a difference is a finding, not silently overridden.
Currency	Currency	Must exist in the ERP.

Posting the bill (confirming it in accounting) is a separate ERP step. The prototype creates the bill in draft state so an accountant can post it; changing this is a decision for the ERP owner.

7.4.4 Submission flow and idempotency

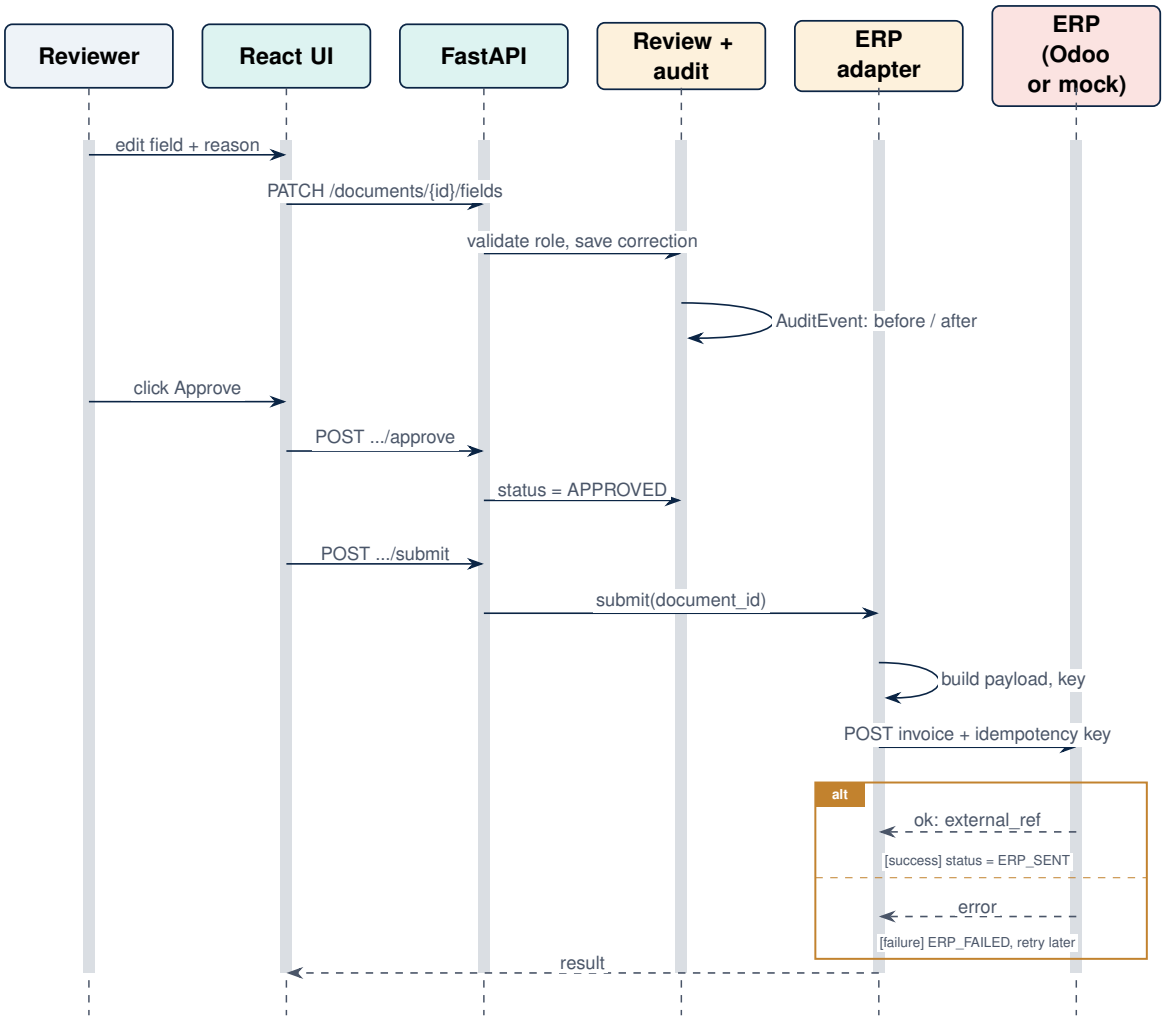


Figure 7.1: *Review, approval and ERP submission.*

1. The service checks that the status is APPROVED or ERP_FAILED and locks the document row.
2. It runs `check_payload`. Any BLOCKING finding stops the submission before anything is sent and the status stays APPROVED.
3. It inserts an `erp_attempts` row (PENDING) with the next attempt number and the key `idp-<document_id>`, then commits.
4. It calls `submit`. On success the attempt becomes SENT with the ERP reference and the document becomes ERP_SENT. On failure the attempt becomes FAILED with a category and the document becomes ERP_FAILED.
5. To our knowledge Odoo has no built-in idempotency-key header. Before creating a bill on a retry, the Odoo adapter therefore searches for an existing bill with the same vendor reference and partner and, if found, returns it as success. This behaviour must be confirmed against the real ERP.

7.4.5 Errors and retries

Category	Example	Handling (values provisional)
AUTH	Wrong or expired ERP key.	No automatic retry; alert the administrator; document stays ERP_FAILED.
VALIDATION	ERP refuses a value.	No automatic retry; reviewer corrects the data, then submits again.

Category	Example	Handling (values provisional)
TIMEOUT	No answer in time.	Retry up to three times with growing delay, same key; the pre-send search prevents a duplicate.
SERVER	ERP error 5xx.	Same as timeout.
DUPLICATE	A bill with this reference exists.	Treated as success and the existing reference is stored.

7.4.6 Mock ERP contract

The mock accepts `POST /vendor-bills` with an `Idempotency-Key` header. A new key returns `201` and a reference. A repeated key returns `200` with the same reference and creates nothing. Switches in the request let tests force a timeout, a `401` and a validation error. The same test suite runs against the mock now and against Odoo when it is available.

8

Traceability, Open Decisions and Next Steps

8.1 Acceptance criteria of the six stories

"Designed" means the design is written in this document or in a companion file. "Checked" means a program verified part of it. It does not mean implemented.

Story	Acceptance criterion	Where	State
US-06	React + Vite as frontend.	Ch. 2	Designed.
US-06	FastAPI as backend API.	Ch. 2	Designed; health endpoints exist and are tested.
US-06	MySQL as relational database.	Ch. 2, 3	Designed; schema checked on MariaDB.
US-06	OCR pipeline and ERP integration in a diagram.	Fig. 2.1, 2.2	Designed.
US-07	Required entities identified.	Ch. 3	Designed (16 tables).
US-07	Tables for documents, extracted fields, review decisions.	Ch. 3	Designed and checked.
US-07	Primary keys, foreign keys, relationships.	Fig. 3.1	Designed and checked (nine negative tests).
US-07	Processing status and audit history.	Ch. 3	Designed and checked (status values, append-only trigger).
US-08	Upload and file validation.	Sec. 4.3	Designed.
US-08	Preprocessing and OCR.	Stages 4–5	Designed; not built.
US-08	Extraction and confidence scoring.	Stages 6–7	Designed; thresholds provisional.
US-08	Human review of uncertain results.	Ch. 4	Designed.
US-08	Approved path to ERP.	Stage 10, Ch. 7	Designed.
US-09	Upload and retrieval endpoints.	Ch. 5	Designed and checked (contract validates).
US-09	Endpoints for fields and review decisions.	Ch. 5	Designed and checked.
US-09	Response formats and error handling.	Ch. 5	Designed.
US-09	Authentication and authorisation.	Ch. 5	Designed; token lifetime open.
US-10	Dashboard sketch.	Fig. 6.2	Designed.
US-10	Upload page.	Fig. 6.3	Designed.
US-10	Extraction results and review screens.	Fig. 6.4	Designed.
US-10	Loading, success and error states.	Ch. 6	Designed.
US-11	Access permissions and credential management.	Ch. 5, 7	Designed.
US-11	File validation and data protection.	Sec. 4.3, Ch. 7	Designed.
US-11	**Target Odoo version and available API.**	Ch. 7	**Not closed.** Options documented; needs the real version (P2-D1).
US-11	Validation and error handling before ERP submission.	Ch. 7	Designed.

8.2 Open decisions

ID	Decision needed	Effect
P2-D1	Which Odoo version and access (cloud, on-premise, test instance)?	Chooses JSON-2 or XML-RPC adapter; closes US-11.
P2-D2	Token lifetime, and whether to add refresh tokens.	Changes the login endpoint and the frontend client.
P2-D3	May a reviewer approve a document they uploaded themselves?	Adds a separation-of-duties check to approval.
P2-D4	Retention period and deletion procedure for documents.	Defines use of `deleted_at` and a clean-up job.
P2-D5	Post vendor bills in the ERP, or leave them in draft?	Changes the final ERP call.
P2-D6	Final thresholds and tolerances.	Set only after the OCR baseline exists.

The open decisions D1 to D9 of the Phase 1 document still apply; in particular the document type scope (D3), the dataset (D6) and the training hardware (D7).

8.3 Risks specific to this phase

Risk	Effect	Mitigation
Odoo version differs from the assumption	ERP adapter must be rewritten.	Adapter interface; mock first; decide P2-D1 early.
Models, migrations and `schema.sql` drift apart	Wrong schema in some environments.	Compare the migrated database with `schema.sql` in a test.
API code drifts from the contract	React and backend disagree.	Compare the generated schema with `openapi.yaml`; contract tests.
Schema tested only on MariaDB	A MySQL 8.0 difference appears late.	Run the file on MySQL 8.0 as the first Phase 3 task.
Design too large for the time available	Late features missing.	Build in the order of Chapter 8; line items and the admin screens can be reduced.

8.4 Next steps

The next phase is the development environment and the working foundation (stories US-12 to US-16). The design above gives it this order of work.

1. Confirm that the Phase 1 backend runs on the developer's PC (health and database checks).
2. Run `schema.sql` on MySQL 8.0 and fix any difference.
3. Write the SQLAlchemy models and the first Alembic revision, including the checks and triggers; compare the migrated schema with `schema.sql`.
4. Install and verify OpenCV and PyTorch, and set up the React and Vite application.
5. Implement sign-in with roles, then upload with file validation, then the status endpoints, each with tests, keeping `openapi.yaml` in step.

9

Extension: Tender and Opportunities Module

9.1 Why this chapter exists

After the first version of this specification the project owner proposed an extension: a buyer publishes a call for tenders (*appel d'offres*), companies receive it and apply with an offer, and the offers are processed and compared. A second idea was a board where companies publish jobs and internships and people apply. This chapter records the decision about scope, the shared design that supports both ideas, and the phase in which each part is built.

Scope decision (to be confirmed by the project owner)

> **Committed (option B):** the core document-processing system of Chapters 2 to 7 plus the tender module. **Stretch goal (option C):** jobs and internships, built on the same tables only after the core and the tender module work end to end. If the stretch goal is not finished, it is described in the final report as designed and partly built.

9.2 Idea and value

Every step of a tender ends in a document: the call, the offers, the award. DocuMind already reads, validates and sends documents to an ERP, so the tender module reuses that engine instead of adding a second product.

1. A company (the buyer) publishes a tender with requirements, a budget and a deadline.
2. Other companies apply before the deadline and attach their offer as a PDF or a scan.
3. DocuMind reads each offer (supplier, total price, currency, delivery time, validity), validates it and compares the offers side by side.
4. The buyer selects a winner. The system then sends a purchase order to the ERP through the same adapter used for invoices.

A company that answers a tender fills in an offer from a template; the system does not write the offer for it.

9.3 Shared design for tenders and jobs

Both ideas have the same pattern: someone publishes a listing, others apply, and each application carries documents. The design therefore uses one set of tables.

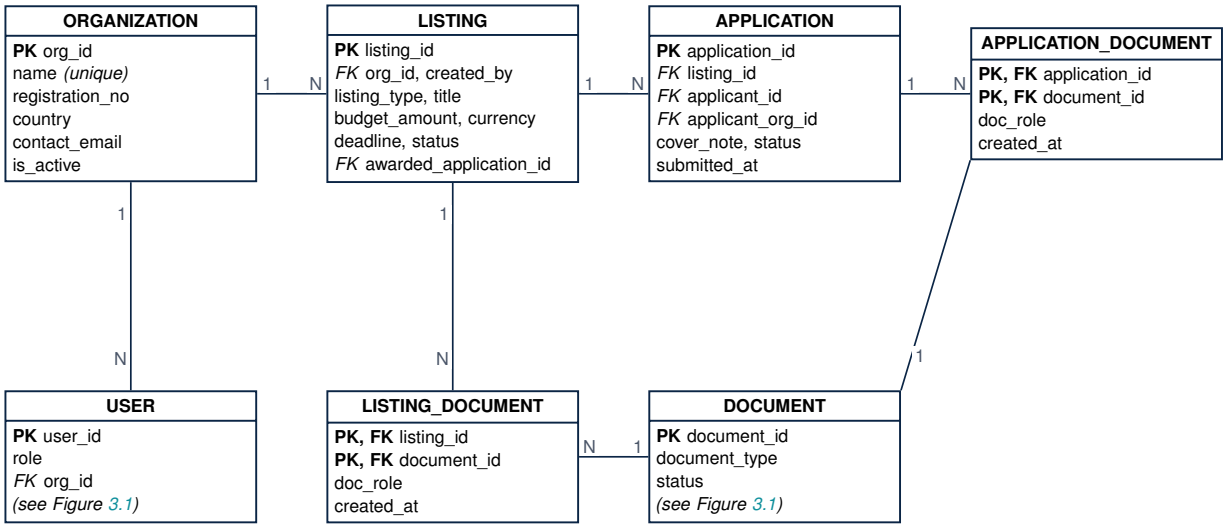


Figure 9.1: *Tables added for the tender module. Each file carried by a listing or an application is an ordinary row of* `documents`*, so it uses the same upload, OCR, extraction and review path as an invoice.*

Table	Purpose
`organizations`	A company profile: name (unique), registration number, country and contact e-mail.
`listings`	A tender, a job or an internship: type, title, description, requirements, budget and currency (tenders), deadline, status, and the winning application once awarded.
`listing_documents`	Files that belong to a listing, such as the call document.
`applications`	One application per person per listing (unique key), with the applying user, the answering company, a cover note and a status.
`application_documents`	The files of an application with their role: offer, CV, cover letter or attachment.

Two existing tables change slightly: `users` gains the roles COMPANY and APPLICANT and an optional `org_id`, and `documents.document_type` gains the values OFFER and CV.

9.4 Roles and permissions

Function	Company	Applicant	Reviewer	Admin
Browse open listings	yes	yes	yes	yes
Create, edit, publish, close or cancel own listing	yes	—	—	yes
Apply to an open listing before its deadline	yes	yes	—	—
See own applications and withdraw them	yes	yes	—	—
See applications received for own listing	yes	—	—	yes
Compare offers of own tender; award it	yes	—	—	yes

The operator role of Chapter 5 stays internal. Four rules protect the data: a company never sees the offers of other companies; an applicant sees only their own applications; the deadline and the OPEN status are checked by the server, not only by the screen; and a listing or application that a user may not see is reported as 404.

9.5 Status models

Object	States	Main transitions
Listing	DRAFT, OPEN, CLOSED, AWARDED, CANCELLED	DRAFT to OPEN on publish; OPEN to CLOSED at the deadline or by the company; CLOSED to AWARDED on award; DRAFT or OPEN to CANCELLED.
Application	SUBMITTED, UNDER_REVIEW, SHORTLISTED, ACCEPTED, REJECTED, WITHDRAWN	SUBMITTED to UNDER_REVIEW when the company opens it; to SHORTLISTED, ACCEPTED or REJECTED by the company; to WITHDRAWN by the applicant while the listing is open.

9.6 API additions

Ten operations are added to `openapi.yaml` (tags Listings and Applications). Each carries an `x-phase` field that says in which phase it is built.

Method	Path	Roles	Phase
GET	`/listings`	all signed in	5
POST	`/listings`	Company, Admin	5
GET	`/listings/{id}`	all signed in	5
PATCH	`/listings/{id}`	Company, Admin	5
POST	`/listings/{id}/applications`	Company, Applicant	5
GET	`/listings/{id}/applications`	Company, Admin	5
PATCH	`/applications/{id}`	Company, Applicant, Admin	5
GET	`/applications/mine`	Company, Applicant	5
GET	`/listings/{id}/comparison`	Company, Admin	10
POST	`/listings/{id}/award`	Company, Admin	12

In the code the route `/applications/mine` must be declared before `/applications/{id}`, otherwise the word “mine” is read as an identifier.

9.7 Link to the ERP

Awarding a tender creates a purchase order for the winning supplier. This is a new method on the `ErpAdapter` interface of Chapter 7, with the same rules: an idempotency key, one transaction that locks the listing, and every attempt stored in `erp_attempts`. Whether Odoo accepts the purchase order in the form assumed here is not verified and depends on the Odoo version (open decision P2-D1). The mock ERP implements it first.

9.8 Where each part is built

Phase	Name in the plan	Tender and opportunities work
2	System design	This chapter, the five new tables, the new roles and the ten API operations (done in this version).
4	Database and backend foundation	Create the five tables and the new roles; models and migration; sign-up and login for companies and applicants; company isolation checks.
5	Document upload and storage	Listing and application endpoints; applicants upload offer files through the existing upload module; deadline and one-application rules.
9	Information extraction	Offer fields: supplier, total price, currency, delivery time, validity. CV fields (name, skills, education) for the stretch goal.
10	Validation and confidence scoring	Offer validation (complete, in budget currency, still valid, deadline respected) and the comparison endpoint.
11	Human review and correction	Review of low-confidence offers before they enter the comparison.
12	ERP integration	Award a tender and send the purchase order to the mock ERP, then Odoo.
13	Web dashboard	Screens: browse listings, apply, my applications, publish a listing, compare offers, award.
14, 15	Testing; security	Tests for isolation, deadlines and duplicate applications; security checks on the new endpoints.
After 13	Stretch goal (option C)	Jobs and internships: listing type JOB and INTERNSHIP in the interface, search, simple CV reading. Only if the core and the tender flow work.

9.9 Open decisions and risks

No.	Decision	Effect
P2-D7	Confirm option B as the committed scope and option C as a stretch goal.	Sets how much of the plan is fixed.
P2-D8	May one user act both as a company and as an applicant, or are accounts separate?	Changes the role model and the login flow.
P2-D9	Are tenders public (any signed-in user) or by invitation only?	Adds an invitation table if by invitation.
P2-D10	Which criteria compare offers (lowest price only, or a weighted score)?	Changes the comparison endpoint.

The main risk is time: the custom OCR model must still be trained and evaluated honestly, so the tender module reuses the existing engine and the jobs board may stay unfinished.

9.10 Verification performed

The extended `schema.sql` loaded into an empty MariaDB 10.11 database (16 tables). A full tender was inserted, then five new invalid operations were refused: a repeated application to one listing, application status DONE, listing type SHOP, application file role PHOTO, and a user with an unknown organization. The nine tests of Section 3.8 were refused again. It has not been run on MySQL 8.0, and the new operations exist only as a validated contract.

A

Database Schema Listing

The complete file `schema.sql`, as it was loaded and tested.

```sql
-- ========================================================================
-- DocuMind AI - Phase 2 database design (MySQL 8.0, InnoDB, utf8mb4)
-- Design reference for the SQLAlchemy models and Alembic migrations of
-- the next phase. Times are stored in UTC. Identifiers are UUID strings.
--
-- Create the database once (outside this file):
-- CREATE DATABASE idp_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
-- Then load this file into it:
-- mysql -u <user> -p idp_db < schema.sql
-- ========================================================================

SET NAMES utf8mb4;

-- ------------------------------------------------------------------------
-- 1. organizations : companies that publish or answer listings (tender module)
-- ------------------------------------------------------------------------
CREATE TABLE organizations (
  org_id CHAR(36) NOT NULL,
  name VARCHAR(200) NOT NULL,
  registration_no VARCHAR(60) NULL, -- trade register or tax number, as given
  country VARCHAR(60) NULL,
  contact_email VARCHAR(255) NULL,
  is_active TINYINT NOT NULL DEFAULT 1,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (org_id),
  UNIQUE KEY uq_organizations_name (name)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 2. users : identity and role (RBAC)
-- ------------------------------------------------------------------------
CREATE TABLE users (
  user_id CHAR(36) NOT NULL,
  email VARCHAR(255) NOT NULL,
  full_name VARCHAR(150) NOT NULL,
  password_hash VARCHAR(255) NOT NULL, -- Argon2id or bcrypt hash, never plaintext
  role VARCHAR(20) NOT NULL, -- OPERATOR | REVIEWER | ADMIN | COMPANY | APPLICANT
  org_id CHAR(36) NULL, -- set for COMPANY users
  is_active TINYINT NOT NULL DEFAULT 1,
  failed_login_count INT NOT NULL DEFAULT 0,
  locked_until DATETIME NULL,
  last_login_at DATETIME NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (user_id),
  UNIQUE KEY uq_users_email (email),
  KEY idx_users_org (org_id),
  CONSTRAINT fk_users_org FOREIGN KEY (org_id) REFERENCES organizations (org_id),
  CONSTRAINT ck_users_role CHECK (role IN ('OPERATOR','REVIEWER','ADMIN','COMPANY','APPLICANT'))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 3. documents : one uploaded source file and its workflow state
-- ------------------------------------------------------------------------
CREATE TABLE documents (
  document_id CHAR(36) NOT NULL,
  uploaded_by CHAR(36) NOT NULL,
  original_filename VARCHAR(255) NOT NULL, -- display only, never used as a path
  storage_key VARCHAR(255) NOT NULL, -- opaque key inside protected storage
  sha256 CHAR(64) NOT NULL,
  mime_type VARCHAR(100) NOT NULL,
  size_bytes BIGINT NOT NULL,
  page_count INT NULL,
  document_type VARCHAR(20) NOT NULL DEFAULT 'INVOICE',
  language VARCHAR(10) NULL, -- fr | en | mixed | unknown
  status VARCHAR(20) NOT NULL DEFAULT 'UPLOADED',
  retry_count INT NOT NULL DEFAULT 0,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  deleted_at DATETIME NULL, -- retention / soft delete
  PRIMARY KEY (document_id),
  UNIQUE KEY uq_documents_storage_key (storage_key),
  KEY idx_documents_status_created (status, created_at),
  KEY idx_documents_sha256 (sha256),
  KEY idx_documents_uploaded_by (uploaded_by, created_at),
  CONSTRAINT fk_documents_user FOREIGN KEY (uploaded_by) REFERENCES users (user_id),
```

```
  CONSTRAINT ck_documents_size CHECK (size_bytes > 0),
  CONSTRAINT ck_documents_type CHECK (document_type IN ('INVOICE','RECEIPT','FORM','OFFER','CV','UNKNOWN')),
  CONSTRAINT ck_documents_status CHECK (status IN
    ('UPLOADED','QUEUED','PROCESSING','NEEDS_REVIEW','APPROVED','REJECTED','ERP_SENT','ERP_FAILED','FAILED'))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ---------------------------------------------------------------------
-- 4. processing_runs : each attempt to process a document (history kept)
-- ---------------------------------------------------------------------
CREATE TABLE processing_runs (
  run_id CHAR(36) NOT NULL,
  document_id CHAR(36) NOT NULL,
  run_number INT NOT NULL, -- 1, 2, 3 ... per document
  status VARCHAR(20) NOT NULL DEFAULT 'RUNNING', -- RUNNING | SUCCEEDED | FAILED
  ocr_engine VARCHAR(20) NOT NULL, -- TESSERACT | CRNN
  model_version VARCHAR(60) NULL,
  preproc_config JSON NULL,
  error_category VARCHAR(40) NULL, -- e.g. UNREADABLE, OCR_ERROR, TIMEOUT
  error_message VARCHAR(500) NULL, -- sanitised, no paths or stack traces
  started_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  ended_at DATETIME NULL,
  PRIMARY KEY (run_id),
  UNIQUE KEY uq_runs_document_number (document_id, run_number),
  CONSTRAINT fk_runs_document FOREIGN KEY (document_id) REFERENCES documents (document_id),
  CONSTRAINT ck_runs_status CHECK (status IN ('RUNNING','SUCCEEDED','FAILED')),
  CONSTRAINT ck_runs_engine CHECK (ocr_engine IN ('TESSERACT','CRNN'))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ---------------------------------------------------------------------
-- 5. extracted_fields : candidate header fields of one run
-- field_name values: supplier_name, supplier_tax_id, invoice_number,
-- invoice_date, due_date, po_reference, subtotal, tax_amount,
-- total_amount, currency (line items live in extracted_line_items)
-- ---------------------------------------------------------------------
CREATE TABLE extracted_fields (
  field_id CHAR(36) NOT NULL,
  run_id CHAR(36) NOT NULL,
  field_name VARCHAR(40) NOT NULL,
  raw_value VARCHAR(1000) NULL, -- exactly what OCR read
  normalized_value VARCHAR(500) NULL, -- parsed date / decimal / code
  reviewed_value VARCHAR(500) NULL, -- final value after human review
  confidence DECIMAL(4,3) NULL, -- 0.000 - 1.000
  page_no INT NULL,
  bbox JSON NULL, -- {"x":..,"y":..,"w":..,"h":..} relative to the page image
  validation_status VARCHAR(10) NOT NULL DEFAULT 'REVIEW',
  validation_message VARCHAR(300) NULL,
  PRIMARY KEY (field_id),
  UNIQUE KEY uq_fields_run_name (run_id, field_name),
  CONSTRAINT fk_fields_run FOREIGN KEY (run_id) REFERENCES processing_runs (run_id),
  CONSTRAINT ck_fields_conf CHECK (confidence IS NULL OR (confidence >= 0 AND confidence <= 1)),
  CONSTRAINT ck_fields_validation CHECK (validation_status IN ('PASS','REVIEW','FAIL'))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ---------------------------------------------------------------------
-- 6. extracted_line_items : invoice table rows of one run (optional field 7)
-- ---------------------------------------------------------------------
CREATE TABLE extracted_line_items (
  item_id CHAR(36) NOT NULL,
  run_id CHAR(36) NOT NULL,
  line_no INT NOT NULL,
  description VARCHAR(500) NULL,
  quantity DECIMAL(18,4) NULL,
  unit_price DECIMAL(18,4) NULL,
  amount DECIMAL(18,2) NULL,
  confidence DECIMAL(4,3) NULL,
  page_no INT NULL,
  bbox JSON NULL,
  PRIMARY KEY (item_id),
  UNIQUE KEY uq_items_run_line (run_id, line_no),
  CONSTRAINT fk_items_run FOREIGN KEY (run_id) REFERENCES processing_runs (run_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ---------------------------------------------------------------------
-- 7. validation_findings : document-level rule results (BR rules)
-- ---------------------------------------------------------------------
CREATE TABLE validation_findings (
  finding_id CHAR(36) NOT NULL,
  run_id CHAR(36) NOT NULL,
  rule_code VARCHAR(40) NOT NULL, -- REQUIRED_FIELD, ARITHMETIC, DUPLICATE, DATE, CURRENCY, ERP_SCHEMA
  severity VARCHAR(10) NOT NULL, -- INFO | WARNING | BLOCKING
  field_names VARCHAR(200) NULL, -- comma-separated field names concerned
  message VARCHAR(500) NOT NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (finding_id),
  KEY idx_findings_run (run_id),
  CONSTRAINT fk_findings_run FOREIGN KEY (run_id) REFERENCES processing_runs (run_id),
  CONSTRAINT ck_findings_severity CHECK (severity IN ('INFO','WARNING','BLOCKING'))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ---------------------------------------------------------------------
-- 8. reviews : human decisions on a document
-- ---------------------------------------------------------------------
CREATE TABLE reviews (
  review_id CHAR(36) NOT NULL,
  document_id CHAR(36) NOT NULL,
  run_id CHAR(36) NOT NULL,
  reviewer_id CHAR(36) NOT NULL,
  decision VARCHAR(10) NOT NULL, -- APPROVED | REJECTED
  reason VARCHAR(500) NULL, -- required when REJECTED (enforced in the API)
  decided_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
  PRIMARY KEY (review_id),
  KEY idx_reviews_document (document_id, decided_at),
  CONSTRAINT fk_reviews_document FOREIGN KEY (document_id) REFERENCES documents (document_id),
  CONSTRAINT fk_reviews_run FOREIGN KEY (run_id) REFERENCES processing_runs (run_id),
  CONSTRAINT fk_reviews_reviewer FOREIGN KEY (reviewer_id) REFERENCES users (user_id),
  CONSTRAINT ck_reviews_decision CHECK (decision IN ('APPROVED','REJECTED'))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ---------------------------------------------------------------------
-- 9. field_corrections : every manual change, original value preserved (BR-5)
-- ---------------------------------------------------------------------
CREATE TABLE field_corrections (
  correction_id CHAR(36) NOT NULL,
  field_id CHAR(36) NOT NULL,
  old_value VARCHAR(500) NULL,
  new_value VARCHAR(500) NULL,
  reason VARCHAR(300) NOT NULL,
  corrected_by CHAR(36) NOT NULL,
  corrected_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (correction_id),
  KEY idx_corrections_field (field_id, corrected_at),
  CONSTRAINT fk_corrections_field FOREIGN KEY (field_id) REFERENCES extracted_fields (field_id),
  CONSTRAINT fk_corrections_user FOREIGN KEY (corrected_by) REFERENCES users (user_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ---------------------------------------------------------------------
-- 10. erp_attempts : every outbound submission (BR-4, BR-6)
-- ---------------------------------------------------------------------
CREATE TABLE erp_attempts (
  attempt_id CHAR(36) NOT NULL,
  document_id CHAR(36) NOT NULL,
  attempt_no INT NOT NULL,
  idempotency_key VARCHAR(80) NOT NULL, -- same key for every retry of one document
  destination VARCHAR(20) NOT NULL, -- MOCK | ODOO
  status VARCHAR(10) NOT NULL DEFAULT 'PENDING', -- PENDING | SENT | FAILED
  http_status INT NULL,
  external_ref VARCHAR(100) NULL, -- id returned by the ERP
  error_category VARCHAR(40) NULL, -- AUTH, VALIDATION, TIMEOUT, SERVER, DUPLICATE
  error_message VARCHAR(500) NULL, -- sanitised
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  finished_at DATETIME NULL,
  PRIMARY KEY (attempt_id),
  UNIQUE KEY uq_erp_document_attempt (document_id, attempt_no),
  KEY idx_erp_idempotency (idempotency_key),
  KEY idx_erp_status (status),
  CONSTRAINT fk_erp_document FOREIGN KEY (document_id) REFERENCES documents (document_id),
  CONSTRAINT ck_erp_destination CHECK (destination IN ('MOCK','ODOO')),
  CONSTRAINT ck_erp_status CHECK (status IN ('PENDING','SENT','FAILED'))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- A document may have at most one successful ERP submission.
-- Enforced in the service layer inside a transaction (row lock on documents)
-- and checked by the retry tests required by objective 09.

-- ---------------------------------------------------------------------
-- 11. audit_events : append-only trail (FR-13)
-- ---------------------------------------------------------------------
CREATE TABLE audit_events (
  event_id BIGINT NOT NULL AUTO_INCREMENT,
  actor_id CHAR(36) NULL, -- NULL for system actions
  document_id CHAR(36) NULL,
  action VARCHAR(50) NOT NULL, -- LOGIN, UPLOAD, STATUS_CHANGE, CORRECT_FIELD, APPROVE, REJECT, ERP_SUBMIT, SETTING_CHANGE
  entity_type VARCHAR(30) NULL,
  entity_id VARCHAR(36) NULL,
  before_json JSON NULL,
  after_json JSON NULL,
  ip_address VARCHAR(45) NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (event_id),
  KEY idx_audit_document (document_id, created_at),
  KEY idx_audit_actor (actor_id, created_at),
  KEY idx_audit_action (action, created_at),
  CONSTRAINT fk_audit_actor FOREIGN KEY (actor_id) REFERENCES users (user_id),
  CONSTRAINT fk_audit_document FOREIGN KEY (document_id) REFERENCES documents (document_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Append-only guarantee at database level (in addition to the application rule).
CREATE TRIGGER trg_audit_no_update BEFORE UPDATE ON audit_events
  FOR EACH ROW SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'audit_events is append-only';
CREATE TRIGGER trg_audit_no_delete BEFORE DELETE ON audit_events
  FOR EACH ROW SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'audit_events is append-only';

-- ---------------------------------------------------------------------
-- 12. system_settings : administrator-editable rules and thresholds (FR-17)
-- ---------------------------------------------------------------------
CREATE TABLE system_settings (
  setting_key VARCHAR(60) NOT NULL,
  setting_value VARCHAR(255) NOT NULL,
  description VARCHAR(255) NULL,
  updated_by CHAR(36) NULL,
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (setting_key),
  CONSTRAINT fk_settings_user FOREIGN KEY (updated_by) REFERENCES users (user_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Provisional defaults. Confidence thresholds MUST be re-tuned on validation data (see the provisional targets in the Phase 1 specification).

INSERT INTO system_settings (setting_key, setting_value, description) VALUES
 ('confidence_accept', '0.90', 'At or above this value a field is shown as accepted (provisional)'),
 ('confidence_review', '0.70', 'Below this value a field requires review (provisional)'),
```

```
  ('arithmetic_tolerance', '0.01', 'Allowed difference for subtotal + tax = total, in currency units'),
  ('max_upload_bytes', '10485760', 'Maximum upload size (10 MB)'),
  ('max_pdf_pages', '10', 'Maximum pages accepted per PDF'),
  ('allowed_mime_types', 'application/pdf,image/png,image/jpeg', 'Accepted upload types');

-- ---------------------------------------------------------------------
-- 13. listings : a tender, a job or an internship (tender module; jobs are a stretch goal)
-- ---------------------------------------------------------------------
CREATE TABLE listings (
  listing_id CHAR(36) NOT NULL,
  org_id CHAR(36) NOT NULL, -- the publishing company
  created_by CHAR(36) NOT NULL,
  listing_type VARCHAR(20) NOT NULL, -- TENDER | JOB | INTERNSHIP
  title VARCHAR(200) NOT NULL,
  description TEXT NOT NULL,
  requirements TEXT NULL,
  location VARCHAR(120) NULL,
  budget_amount DECIMAL(15,2) NULL, -- tenders only
  currency CHAR(3) NULL,
  deadline DATETIME NOT NULL, -- UTC
  status VARCHAR(20) NOT NULL DEFAULT 'DRAFT',
  awarded_application_id CHAR(36) NULL, -- winner of a tender (set on award)
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (listing_id),
  KEY idx_listings_status_deadline (status, deadline),
  KEY idx_listings_type (listing_type, status),
  KEY idx_listings_org (org_id, created_at),
  CONSTRAINT fk_listings_org FOREIGN KEY (org_id) REFERENCES organizations (org_id),
  CONSTRAINT fk_listings_user FOREIGN KEY (created_by) REFERENCES users (user_id),
  CONSTRAINT ck_listings_type CHECK (listing_type IN ('TENDER','JOB','INTERNSHIP')),
  CONSTRAINT ck_listings_status CHECK (status IN ('DRAFT','OPEN','CLOSED','AWARDED','CANCELLED')),
  CONSTRAINT ck_listings_budget CHECK (budget_amount IS NULL OR budget_amount >= 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ---------------------------------------------------------------------
-- 14. listing_documents : files that belong to a listing (call for tender, job description)
-- ---------------------------------------------------------------------
CREATE TABLE listing_documents (
  listing_id CHAR(36) NOT NULL,
  document_id CHAR(36) NOT NULL,
  doc_role VARCHAR(20) NOT NULL DEFAULT 'ATTACHMENT', -- CALL_DOCUMENT | ATTACHMENT
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (listing_id, document_id),
  CONSTRAINT fk_ld_listing FOREIGN KEY (listing_id) REFERENCES listings (listing_id),
  CONSTRAINT fk_ld_document FOREIGN KEY (document_id) REFERENCES documents (document_id),
  CONSTRAINT ck_ld_role CHECK (doc_role IN ('CALL_DOCUMENT','ATTACHMENT'))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ---------------------------------------------------------------------
-- 15. applications : one company offer or one candidate application for a listing
-- ---------------------------------------------------------------------
CREATE TABLE applications (
  application_id CHAR(36) NOT NULL,
  listing_id CHAR(36) NOT NULL,
  applicant_id CHAR(36) NOT NULL, -- the user who applies
  applicant_org_id CHAR(36) NULL, -- the answering company (tenders)
  cover_note TEXT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'SUBMITTED',
  submitted_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (application_id),
  UNIQUE KEY uq_applications_once (listing_id, applicant_id), -- one application per person per listing
  KEY idx_applications_listing (listing_id, status),
  KEY idx_applications_applicant (applicant_id, submitted_at),
  CONSTRAINT fk_app_listing FOREIGN KEY (listing_id) REFERENCES listings (listing_id),
  CONSTRAINT fk_app_applicant FOREIGN KEY (applicant_id) REFERENCES users (user_id),
  CONSTRAINT fk_app_org FOREIGN KEY (applicant_org_id) REFERENCES organizations (org_id),
  CONSTRAINT ck_app_status CHECK (status IN ('SUBMITTED','UNDER_REVIEW','SHORTLISTED','ACCEPTED','REJECTED','WITHDRAWN'))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

ALTER TABLE listings
  ADD CONSTRAINT fk_listings_awarded FOREIGN KEY (awarded_application_id) REFERENCES applications (application_id);

-- ---------------------------------------------------------------------
-- 16. application_documents : the files an application carries (offer PDF, CV, cover letter)
-- Each file is an ordinary row of documents, so it goes through OCR, extraction and review.
-- ---------------------------------------------------------------------
CREATE TABLE application_documents (
  application_id CHAR(36) NOT NULL,
  document_id CHAR(36) NOT NULL,
  doc_role VARCHAR(20) NOT NULL, -- OFFER | CV | COVER_LETTER | ATTACHMENT
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (application_id, document_id),
  CONSTRAINT fk_ad_application FOREIGN KEY (application_id) REFERENCES applications (application_id),
  CONSTRAINT fk_ad_document FOREIGN KEY (document_id) REFERENCES documents (document_id),
  CONSTRAINT ck_ad_role CHECK (doc_role IN ('OFFER','CV','COVER_LETTER','ATTACHMENT'))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```